1. Name & ID

The University of the West Indies

COMP 3613 · Assignment 2

MyAdvisor

Student–Advisor degree planning and course approval system

Student name	Cameron Ramesar
University ID	816044668
Project	MyAdvisor
Repository	https://github.com/Camcoon/MyAdvisorWithTests
Document prepared	4 November 2026

Approved fictional coursework configuration — 93 credits

Requirement category	Credits
Core Courses	36
General Education	18
Electives	12
Major Requirements	18
Additional Requirements	9
Total	**93**

The 93-credit configuration is approved fictional coursework demonstration data. It is not official UWI curriculum data.

This document presents the Assignment 2 project description, use case diagram, entity relationship diagram, application wireframe, test plans, implementation evidence, test reports and UAT summary.

Contents

Page numbers refer to the printed pages of this document. The eight lecturer-required sections are followed by the UAT execution summary and two appendices.

1. Name & ID . . . 1
2. Project Description . . . 3
3. Use Case Diagram . . . 5
4. Entity Relationship Diagram . . . 6
4.1 Readable entity keys and relationships . . . 8
5. Application Wireframe . . . 9
6. Test Plans . . . 14
6.1 User Acceptance Test Plan . . . 14
6.2 Unit Test Plan . . . 23
6.3 Integration Test Plan . . . 27
7. Implementation . . . 31
7.1 GitHub Repository . . . 31
7.2 Unit Tests . . . 31
7.3 Integration Tests . . . 32
7.4 API Tests — Postman . . . 32
8. Reports . . . 34
8.1 Unit Test Report . . . 34
8.2 Integration Test Report . . . 38
8.3 Postman Collection Run Report . . . 42
9. UAT Execution Summary . . . 46
Appendix A — Testing Limitations and Future Improvements . . . 57
Appendix B — Submission Checklist . . . 58

2. Project Description

MyAdvisor is a web-based academic planning prototype built for the COMP 3613 Assignment 2 coursework. It helps a Student understand degree progress, choose an eligible semester plan, and receive an Advisor decision in one connected workflow. The system addresses the difficulty of reconciling completed courses, in-progress courses and proposed selections across separate records, and of communicating plan decisions without a shared review queue.

The Student role (regular_user) owns degree progress and semester plans. The Advisor role (admin) reviews the shared Pending queue and records a decision. Student identity comes from the authenticated account, not from a submitted identifier; Advisor decisions update the plan status, reviewer and feedback.

Workflow	Purpose and implemented behaviour
WF1 — Track Degree Progress (Student)	View overall progress; completed, in-progress and remaining credits; requirement categories; recent academic activity.
WF2 — Plan Semester Course Selections (Student)	Search and filter the catalogue; inspect eligibility; select courses; review and submit a plan; read feedback; revise, remove permitted courses and cancel or replace plans.
WF3 — Review and Approve Course Plans (Advisor)	Inspect the Pending queue, Student and course details; approve, reject or request changes with optional feedback.

Semesters use the nine fixed Year 1–3 / semester 1–3 options, with semester 3 designated summer. Only Pending plans appear in Advisor Review. Pending and Approved plans lock standard selection; Rejected and Changes Requested plans are editable. Cancellation is allowed for Pending, Rejected and Approved plans and is absent for Changes Requested and Cancelled plans.

2.1 Technologies and architecture

Technology	Role in the implementation
Python 3.14, FastAPI, Pydantic	HTTP application, request validation and JSON API contracts.
SQLModel, SQLAlchemy, SQLite	Models, repository persistence and isolated local test databases.
Jinja2, Bootstrap/AdminLTE, JavaScript	Server-rendered Student and Advisor pages; selection, review and confirmation interactions.
pwdlib, PyJWT, signed cookies	Password verification, signed login token, current-role and ownership checks.
pytest, TestClient, Postman	Isolated unit decisions, integrated HTTP/database checks and the automated API collection.

The application uses a layered structure: routers handle HTTP and template/JSON responses, services hold business rules, repositories own persistence, and SQLModel models define the data contract. Templates render the Student and Advisor workspaces. JSON endpoints expose the same service layer used by the browser workflows.

Assignment 2 scope. The assignment extends the Assignment 1 workflows with explicit JSON API contracts, unit and integration test plans and pytest implementations, a 90-request Postman collection, manual UAT specifications and execution records, screenshots, reports and requirements traceability.

2.2 Approved credit configuration

Requirement category	Approved credits
Core Courses	36
General Education	18
Electives	12
Major Requirements	18
Additional Requirements	9
Total	**93**

This 36/18/12/18/9 allocation is fictional coursework demonstration data approved for the project. It is not an official UWI curriculum or a claim about any real programme.

2.3 Scope, assumptions and boundaries

Aspect	Project decision
Data	All accounts, courses, programmes, grades and credit values are fictional demonstration data. They do not describe a real UWI programme or student record.
Runtime	The application runs locally against SQLite. No production deployment, hosting or live user data is part of this submission.
Authentication	Login uses the starter session cookie with a signed token; role checks read the current database role at request time.
Planning rules	Nine fixed Year 1–3 semester options; one active Pending or Approved plan per semester; Rejected and Changes Requested plans are revised and resubmitted under the same plan ID.
Out of scope	Official curriculum validation, production security hardening, mobile layout review, concurrent multi-user operation and PostgreSQL behaviour. Appendix A records these limits.

3. Use Case Diagram

The original Assignment 1 use case diagram defines the project actors and use cases. It is reproduced without redrawing. The Student tracks degree progress, plans semester course selections, views the catalogue and cancels semester plans. The Advisor reviews course plans and can request changes. Catalogue viewing is included in the planning and review workflows; Request Changes extends review. Authentication supports these workflows and is tested separately.

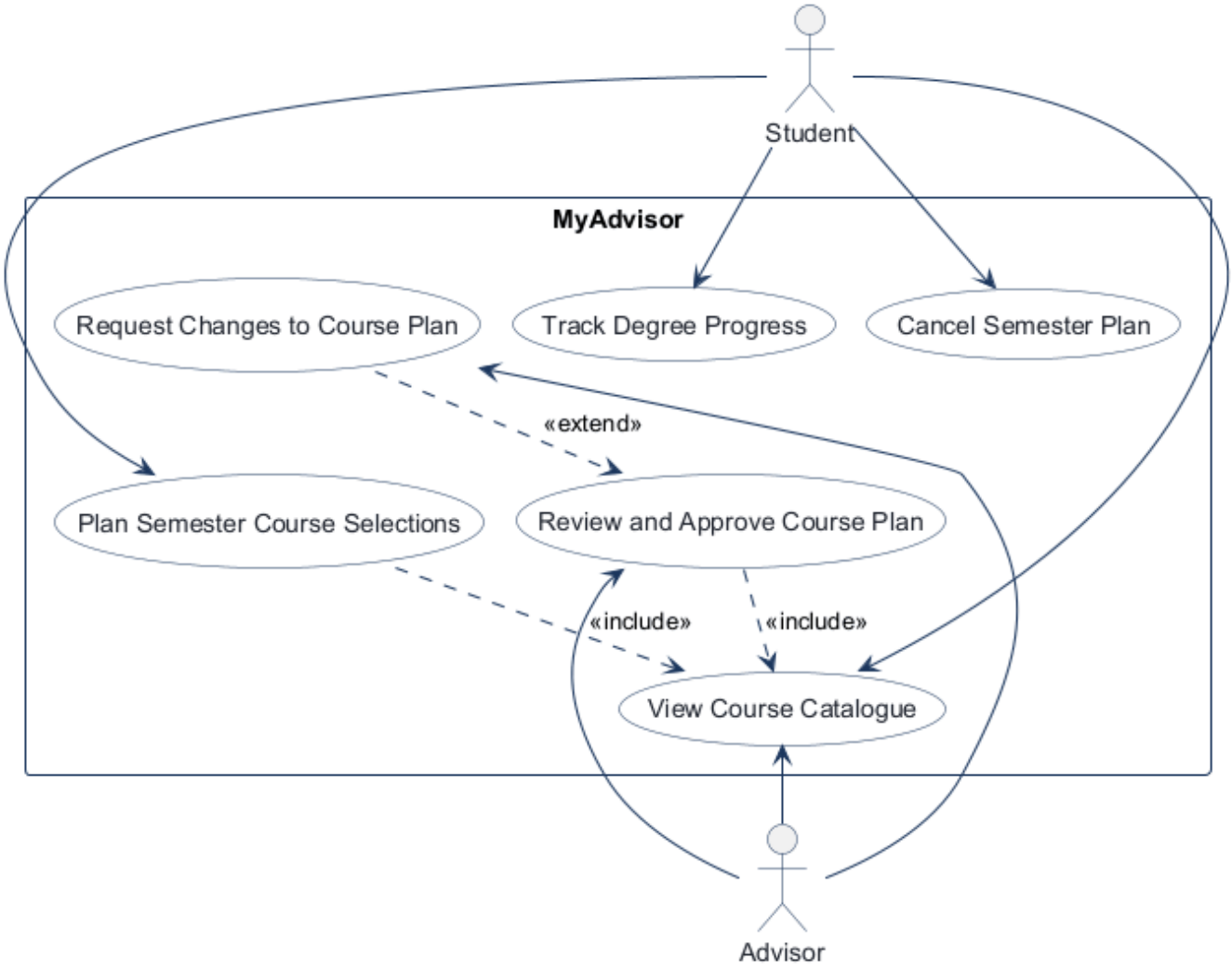


Figure 3.1. Original Assignment 1 MyAdvisor use case diagram, reproduced from docs/diagrams/use-case.png without redrawing.

Actors. Student (regular_user) and Advisor (admin). **Included use cases.** View Course Catalogue is included by Plan Semester Course Selections and Review Course Plans because both workflows need course details. **Extending use case.** Request Changes extends Review Course Plans when the Advisor needs more information before a decision. Registration and login are supporting authentication paths, not part of WF1–WF3.

4. Entity Relationship Diagram

The original seven-entity ERD defines DegreeProgram, DegreeRequirement, User, Course, SemesterPlan, PlanCourse and StudentCourse. A programme defines requirements and optionally enrols users; a Student creates plans and has course records; an optional Advisor reviews each plan. PlanCourse joins plans to courses, and StudentCourse joins users to academic course records.

Relationship (original Mermaid)	Reads as
DEGREE_PROGRAM o|--o{ USER : enrolls	A user has zero or one programme; a programme can enrol many users.
DEGREE_PROGRAM ||--o{ DEGREE_REQUIREMENT : defines	Each requirement belongs to exactly one programme.
USER ||--o{ SEMESTER_PLAN : creates	Each plan has exactly one Student creator.
USER o|--o{ SEMESTER_PLAN : reviews	A plan has zero or one reviewing Advisor; an Advisor can review many plans.
USER ||--o{ STUDENT_COURSE : completes	An academic record joins one Student to one course with status, grade and date.
COURSE ||--o{ STUDENT_COURSE : appears_in	Each academic record refers to exactly one course.
COURSE ||--o{ PLAN_COURSE : appears_in	Each selected-plan row refers to exactly one course.
SEMESTER_PLAN ||--o{ PLAN_COURSE : contains	PlanCourse implements the plan-to-course association.

All seven id fields are primary keys. Unique pairs constrain programme/category, student/course and plan/course. Foreign-key fields retain their original names. The password field names stored data; no password value is reproduced.

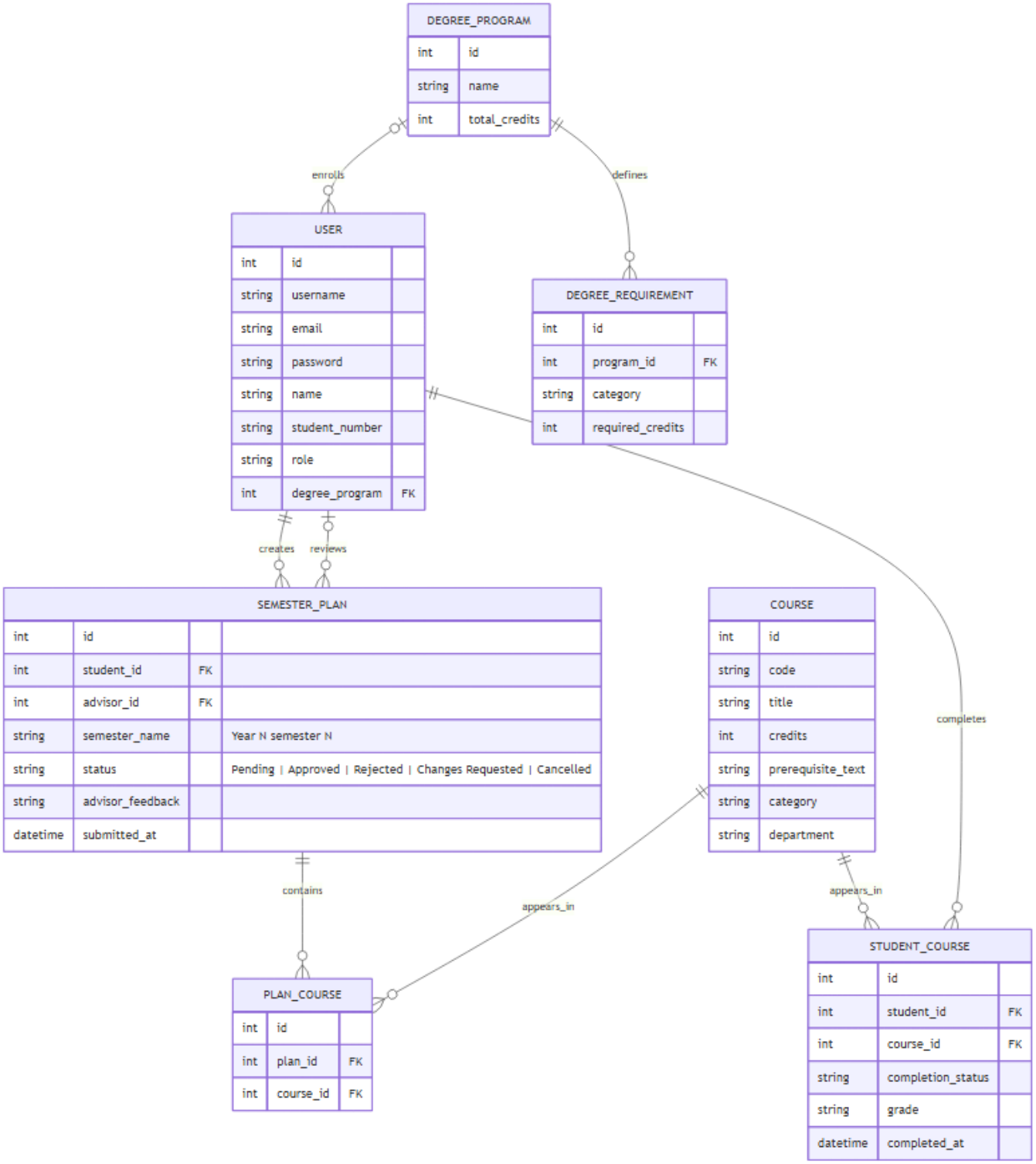


Figure 4.1. Original MyAdvisor ERD image extracted losslessly from the Assignment 1 report (docs/report.pdf, page 3). The readable keyed rendering follows in Figure 4.2.

All seven id fields are primary keys. Unique pairs constrain programme/category, student/course and plan/course. Foreign-key fields retain their original names. The password field names stored data; no password value is reproduced.

4.1 Readable entity keys and relationships

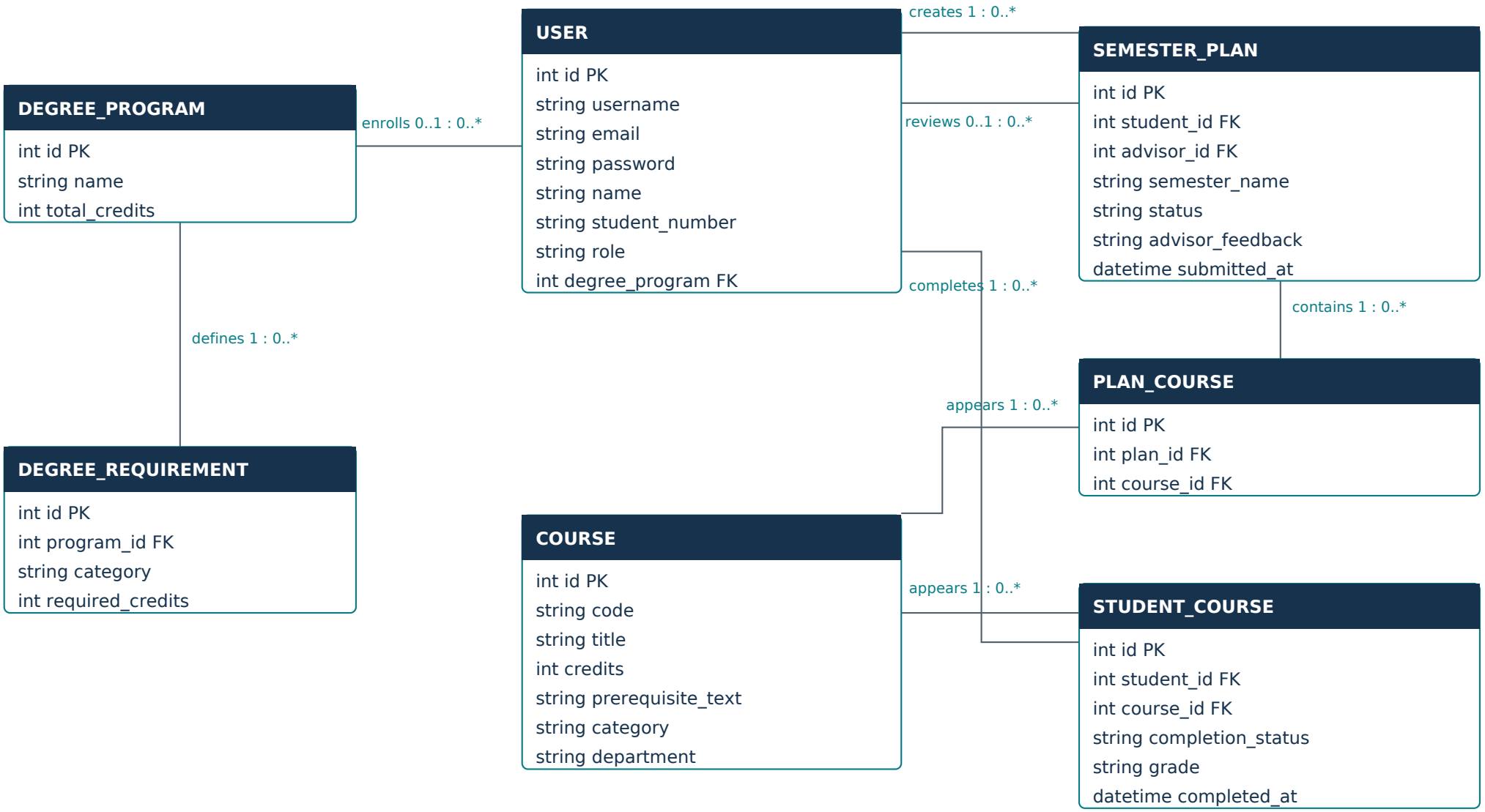


Figure 4.2. Enlarged vector rendering of the same original seven-entity Mermaid ERD. All original fields and relationships are retained; PK labels are added from app/models/user.py and app/models/advising.py. Numeric edge labels state source:target cardinality; “appears” abbreviates “appears_in”.

5. Application Wireframe

The original Assignment 1 three-panel wireframe is reproduced unmodified. W1 is the Student Degree Progress view, W2 is Student Semester Planning, and W3 is Advisor Plan Review. Sample credits, dates, course labels and the eight-digit Student ID in the mockup are illustrative, not execution results.

Wireframe region	Implemented surface	Related evidence
W1 — Degree Progress	Student Degree Progress: overall progress, credit cards, requirement breakdown and recent activity.	UAT-01, UAT-18
W2 — Semester Planning	Plan Courses: catalogue search and filters, eligibility, selection count, Review Selected Courses and Submit Plan.	UAT-02–05, UAT-08–17
W3 — Advisor Review	Advisor workspace: shared Pending queue, Student and credit details, Approve, Reject and Request Changes.	UAT-06–11
W4 — Messages (extension)	Student and Advisor Messages showing decisions and feedback; not a separate wireframe panel.	UAT-07, UAT-08, UAT-11
W5 — Cancellation (extension)	Cancellation, replacement and resubmission controls added to W2/W3 during the build.	UAT-14–17

Reading the wireframe against the implementation. The mockup fixed the information architecture rather than a pixel-perfect layout. The implemented Student pages add session-aware navigation, an explicit selection counter, a review-and-submit step and status-aware actions. The Advisor page adds the shared Pending queue and the decision form used by WF3. Where the mockup shows illustrative 120-credit totals and calendar-year labels, the implemented screens show the approved 93-credit configuration and the nine fixed Year 1–3 semester labels.

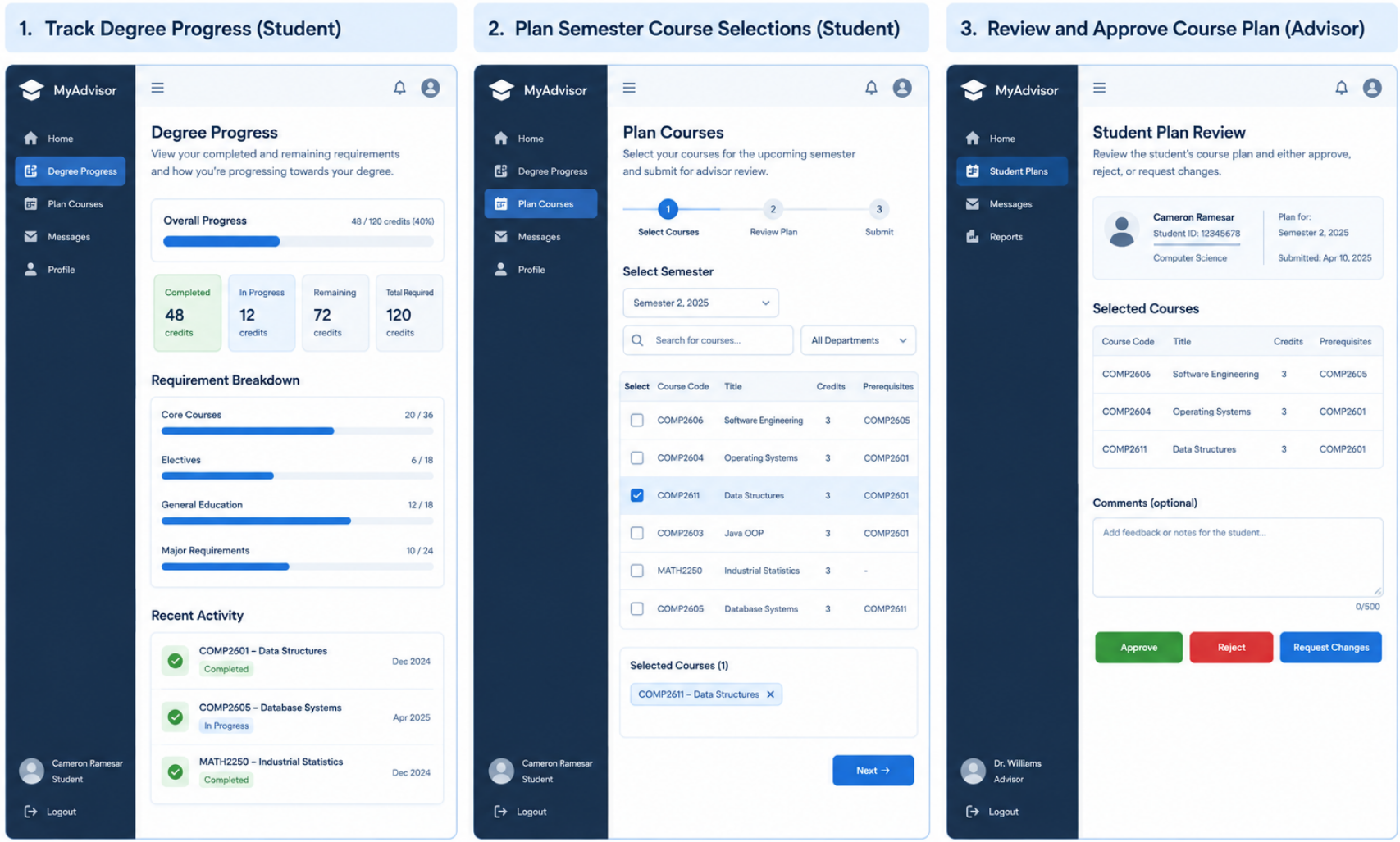


Figure 5.1. Original three-panel Assignment 1 wireframe, unmodified. W1: Student Degree Progress; W2: Student Semester Planning; W3: Advisor Plan Review.

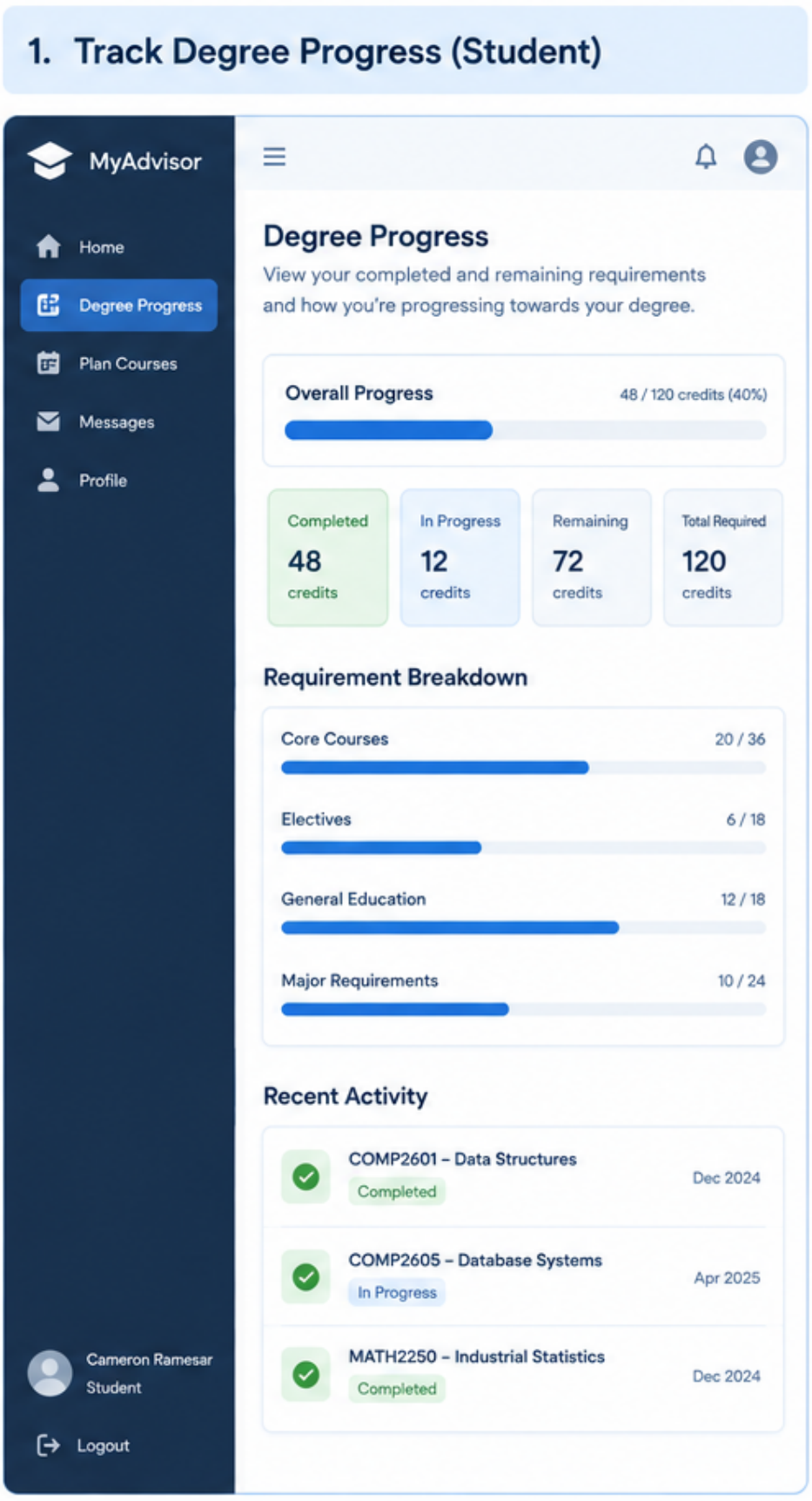


Figure 5.2. Wireframe detail: W1 — Student Degree Progress. Enlarged crop of the original panel. W1 maps to UAT-01; W2 to UAT-02–05 and UAT-08–17; W3 to UAT-06–11. Login, cancellation and resubmission extensions are documented in Section 6.1.

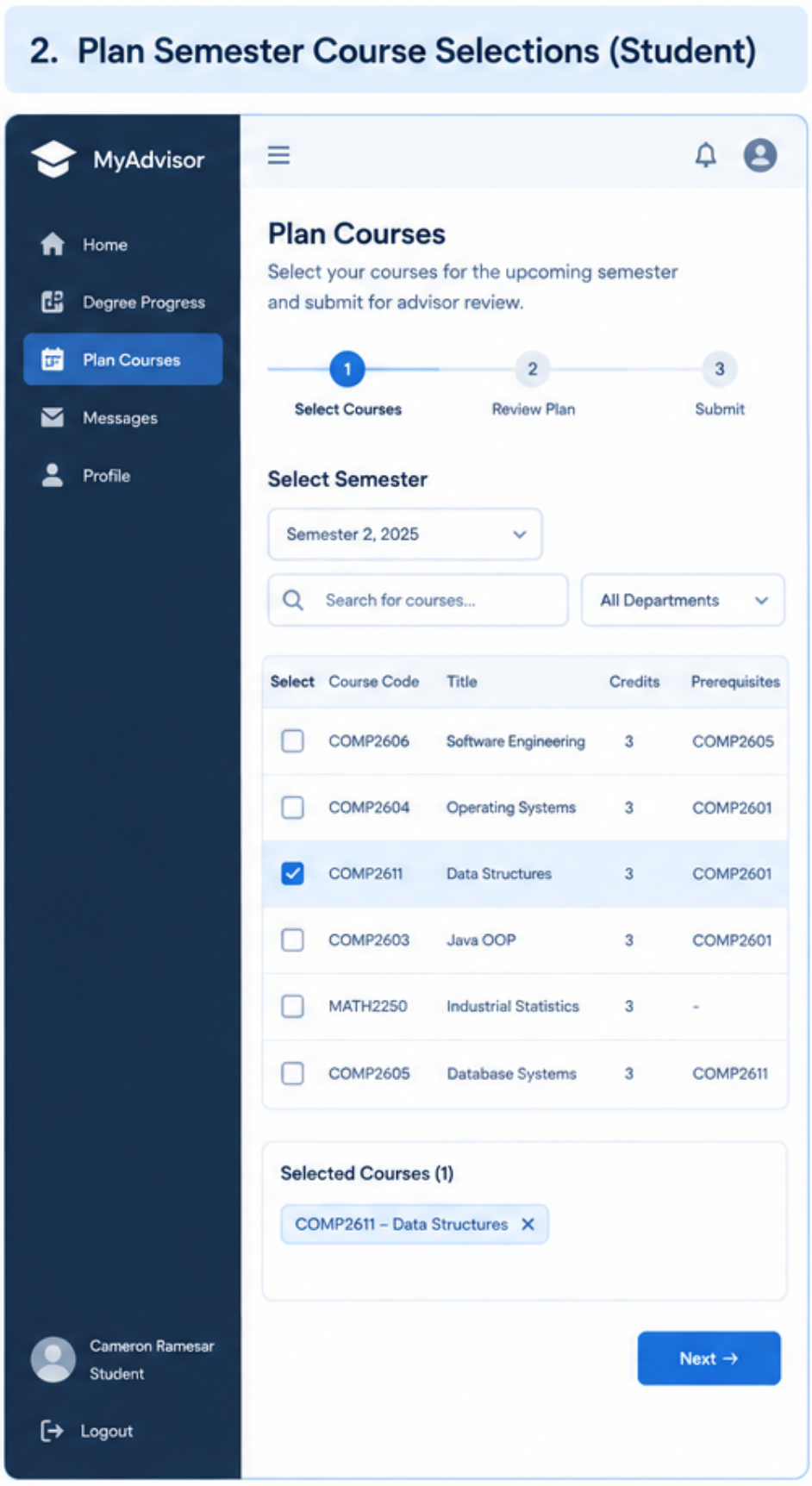


Figure 5.3. Wireframe detail: W2 — Student Semester Planning. Enlarged crop of the original panel. W1 maps to UAT-01; W2 to UAT-02–05 and UAT-08–17; W3 to UAT-06–11. Login, cancellation and resubmission extensions are documented in Section 6.1.

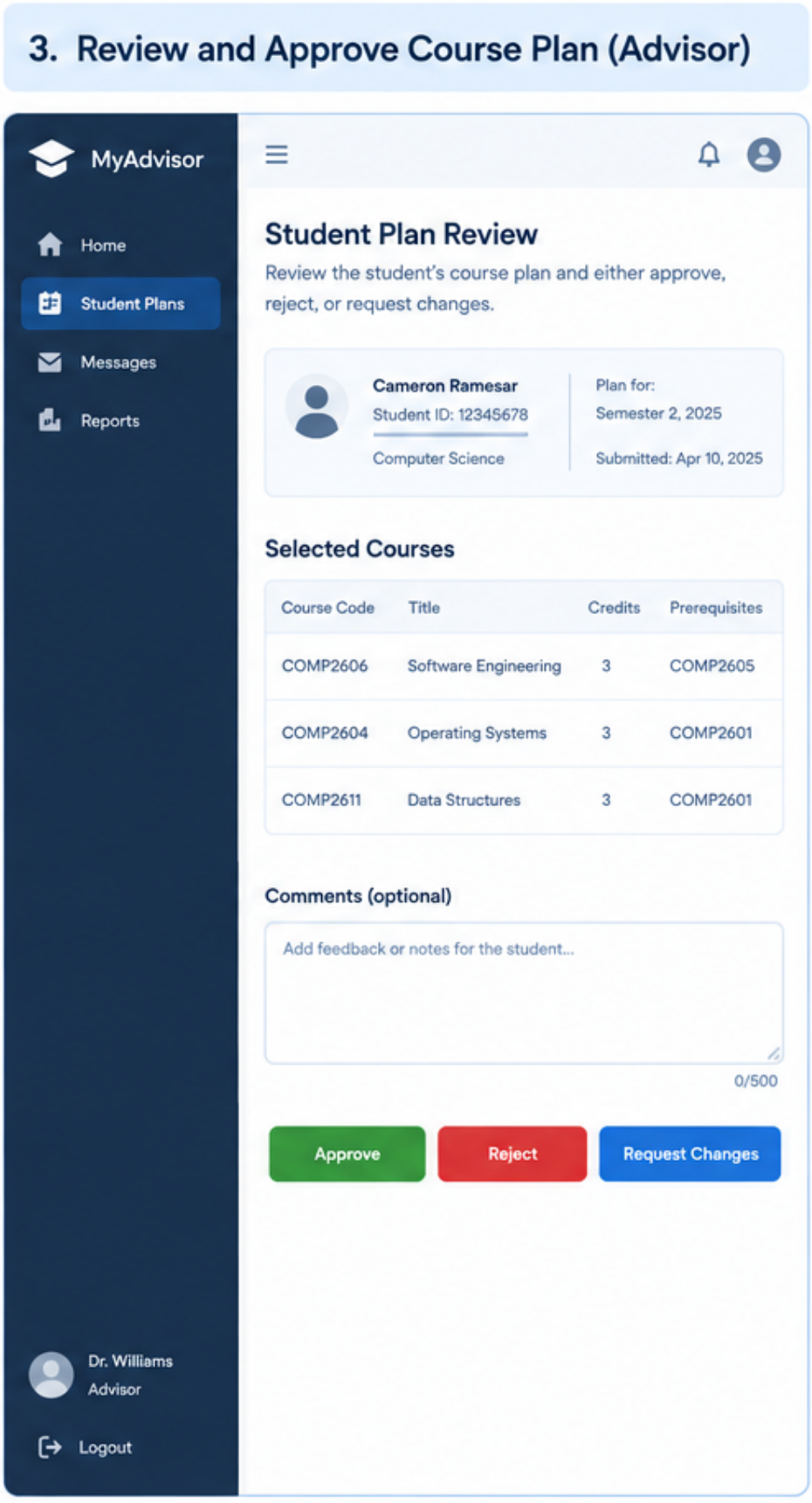


Figure 5.4. Wireframe detail: W3 — Advisor Plan Review. Enlarged crop of the original panel. W1 maps to UAT-01; W2 to UAT-02–05 and UAT-08–17; W3 to UAT-06–11. Login, cancellation and resubmission extensions are documented in Section 6.1.

W4 denotes Student Messages and feedback; W5 denotes the cancellation and resubmission extensions to W2/W3. Neither is an additional original wireframe panel.

6. Test Plans

6.1 User Acceptance Test Plan

Objective and scope. Verify that Students can understand degree progress and complete eligible semester planning, and that Advisors can review and return decisions. The tables below record the planned browser tasks and expected outcomes. Observed results and statuses are reported separately in Section 9; automated executions are reported in Section 8.

Plan control	Specification
Actors	Student (regular_user) and Advisor (admin); separate browser profiles to isolate cookies.
Environment	Prepared disposable SQLite database; current local application; JavaScript enabled; base URL http://127.0.0.1:5002.
Preparation	Use the safe local setup in the UAT execution record; do not reset an existing database. Confirm empty A/B/C semesters and the expected demo identities.
Execution order	UAT-18 → UAT-01–05 → UAT-19 → UAT-06–17 → UAT-20. Numeric identifiers are retained.
References	W1/W2/W3 are the panels in Figures 5.2/5.3/5.4. W4 means Student Messages/feedback; W5 means the cancellation/resubmission extensions to W2/W3.
Recording	Record the observed outcome, actual evidence filename, deviation and remaining checks. PASS only when the complete required outcome is observed; observed primary outcomes with incomplete substeps are Executed — Partially Verified; FAIL only for an observed mismatch.

6.1.1 Planned UAT data and baseline

Item	Planned value
Semesters A / B / C	Year 1 semester 1 / Year 1 semester 2 / Year 1 semester 3.
Courses E1 / E2 / E3	COMP 1201 Object-Oriented Programming (requires completed COMP 1101); MATH 2250 Industrial Statistics; ENGL 1000 Academic Writing. Each is 3 credits; E2 and E3 have no prerequisites.
Academic record	COMP 1101 and MATH 1200 Completed, 3 credits each; COMP 1600 In Progress, 3 credits. COMP 2100 is blocked because COMP 1600 is not completed.
Degree requirements	Core Courses 36; General Education 18; Electives 12; Major Requirements 18; Additional Requirements 9; total 93 credits.
Baseline totals	6 completed; 3 in progress; 84 remaining; 93 required; 6% overall progress.
Category progress	Core 3/36 (33 remaining); General Education 3/18 (15 remaining); Electives 0/12; Major Requirements 0/18; Additional Requirements 0/9.

The seeded demo identities (Cameron Ramesar, Student ID 12345678, Dr. Williams) and the 36/18/12/18/9 allocation are fictional coursework data. The author's University ID is 816044668 and is shown on the cover.

UAT-01 — View degree progress

Field	Planned specification
Actor / wireframe	Student; W1: Overall Progress, credit cards, Requirement Breakdown, Recent Activity.
Objective	Understand completed, current, and remaining degree requirements.
Preconditions	Local setup and UAT-18 completed; Student signed in; compare displayed data with the documented seed baseline.
Test data	Student demo account; baseline progress and category values above.
Procedure	1. In the signed-in Student window click **Degree Progress** in the sidebar. 2. Read Overall progress, all four credit cards, every Requirement breakdown row, and Recent course activity. Capture the displayed values. 3. Refresh the page and compare the values again.
Expected result	Student workspace opens. Credits are 6 completed, 3 in progress, 84 remaining, and 93 required; overall progress is 6/93 (6%). The category breakdown is Core 3/36 (33 remaining), General Education 3/18 (15 remaining), Electives 0/12, Major Requirements 0/18 and Additional Requirements 0/9. Recent activity shows the three recorded courses with their correct statuses. Refresh preserves the values.

UAT-02 — Browse, search, and filter available courses

Field	Planned specification
Actor / wireframe	Student; W2: semester dropdown, search, department filter, course list.
Objective	Find courses and understand their credits, prerequisites, and eligibility.
Preconditions	UAT-01 completed; Student signed in; A has no plan.
Test data	A; E1/E2; search COMP 1201, Industrial Statistics, and ZZZ-UAT-NO-COURSE.
Procedure	1. Click **Plan Courses**. Open **Semester** and inspect the nine Year 1–3/semester 1–3 options; choose **Year 1 semester 1**. 2. Scroll the course list and inspect course codes, titles, credits, departments, prerequisites, and eligibility messages. 3. Enter COMP 1201 in **Search by code or course name**; inspect the matching row. 4. Replace the search with Industrial Statistics; inspect MATH 2250. 5. Clear the search and choose **Mathematics** in the department dropdown; inspect the remaining rows. 6. With Mathematics still selected, search COMP 1201; inspect the no-match message, then click **Clear Filters**. 7. Search ZZZ-UAT-NO-COURSE; inspect the no-match message and click **Clear Filters** again.
Expected result	The chosen semester loads, and semester 3 is identified as summer. E1 and E2 show their documented details. Search matches code or title, and the department filter restricts results. Combined incompatible filters and the nonexistent search show “No courses match your search and department filters.” Clear Filters restores the catalog and All departments. No plan is submitted.

UAT-03 — Select only eligible courses and review the selection

Field	Planned specification
Actor / wireframe	Student; W2: checkboxes, Selected Courses, Next/Review Plan.
Objective	Prevent an empty or ineligible selection and check the proposed credit load.
Preconditions	UAT-02 completed; A selected; filters clear; no boxes selected.
Test data	E1 and E2 (6 credits); ineligible baseline examples.
Procedure	1. Observe **0 courses selected** and attempt to click **Review Selected Courses**. 2. Locate COMP 1101, COMP 1600, and COMP 2100, scrolling as necessary. Read their eligibility messages and attempt to select each checkbox. 3. Select COMP 1201 and MATH 2250. Check the count, then click **Review Selected Courses**. 4. Read the two reviewed courses, their prerequisites, and **Total selected credits**. 5. Click **Back to course selection**, uncheck MATH 2250, and check the count. Recheck it and click **Review Selected Courses** again.
Expected result	Empty selection cannot advance. The three ineligible checkboxes are disabled with the documented reasons; an in-progress prerequisite does not count as completed. Only E1/E2 are selected. The count moves 0 → 2 → 1 → 2; review shows exactly E1/E2 and 6 credits. Returning to selection retains choices. Nothing is submitted yet; leave the browser on review for UAT-04.

UAT-04 — Submit semester plan A

Field	Planned specification
Actor / wireframe	Student; W2: Review Plan and Submit.
Objective	Send the reviewed selection to an Advisor.
Preconditions	UAT-03 left E1/E2 on the review step for A; no existing A plan.
Test data	A; E1/E2, 6 credits.
Procedure	1. Confirm E1/E2 and 6 credits on **Review your selection**. 2. Click **Submit Plan for Advisor Review** once. 3. Read the confirmation, semester/status banner, and **Selected courses for Year 1 semester 1** list. 4. Refresh the page. Attempt to alter a course checkbox and click **Review Selected Courses**.
Expected result	“Semester plan submitted for advisor review.” appears. A is Pending with exactly E1/E2 and is awaiting advisor review. Refresh preserves the submission. Standard selection controls and the review button are disabled for the submitted semester, preventing another submission through this form. Pending-plan Remove/Cancel controls remain available as separate permitted actions; leave them untouched here.

UAT-05 — View submission status

Field	Planned specification
Actor / wireframe	Student; W2/W4: submitted plan, Home, Messages navigation.
Objective	Find the current submission state after navigating away and signing in again.
Preconditions	UAT-04 completed; only A has been submitted.
Test data	A Pending; E1/E2, 6 credits.
Procedure	1. Click **Home** and read **Latest semester plan**. 2. Click **Messages** and find the **Year 1 semester 1 plan** entry. 3. Click **Sign out**, open /login, enter the Student credentials, and click **Sign in**. 4. Click **Plan Courses**, choose **Year 1 semester 1**, and inspect its status and selected courses.
Expected result	Home shows A Pending, 2 courses, 6 credits, and a waiting-for-review message. Messages identifies the same semester as Pending. Signing in again preserves A and its selection; no decision is implied before the Advisor acts.

UAT-06 — Advisor reviews the submitted plan

Field	Planned specification
Actor / wireframe	Advisor; W3: Student Plans, student summary, Selected Courses table.
Objective	Inspect the correct Student's submission and credit load before deciding.
Preconditions	UAT-05 and UAT-19 completed; A Pending; Advisor signed in in its separate browser profile.
Test data	Advisor demo account; Student identity recorded during setup; A; E1/E2.
Procedure	1. Confirm the separate Advisor window shows Dr. Williams and Advisor workspace after UAT-19. 2. Click **Advisor Review** in the sidebar. 3. Find the card matching the recorded Student identity and **Year 1 semester 1**. 4. Inspect student ID, program, submitted date, Pending status, course count, each course row, prerequisites, and **Total requested credits**. 5. Locate **Comments (optional)**, its 0/500 counter, **Approve Plan**, **Reject Plan**, and **Request Changes**. Do not click a decision yet.
Expected result	The Advisor sees A in the queue with the correct Student identity, Computer Science program, submission date matching execution, E1/E2, 2 courses, and 6 total credits. All three decision controls are present. Reading the card leaves the plan Pending. Match by Student plus semester because the queue may contain other students.

UAT-07 — Request changes and deliver feedback

Field	Planned specification
Actor / wireframe	Advisor (Student verifies delivery in the named Student steps); W3: Comments and Request Changes; W4/W5: feedback and returned plan.
Objective	Return a plan with clear revision instructions that the Student can read.
Preconditions	UAT-06 completed; A still Pending.
Test data	Feedback: Replace MATH 2250 with ENGL 1000, then resubmit.
Procedure	1. In the Advisor's A card, enter the exact feedback above into **Comments (optional)**. Observe the character counter. 2. Click **Request Changes** and read the confirmation; refresh **Advisor Review**. 3. Click Advisor **Messages** and locate the Student/A decision. 4. In the Student window click **Messages**, refreshing if already there, and locate A. 5. Read the feedback and click **Review and update plan**. Inspect the semester, status, selected courses, and available actions.
Expected result	Advisor confirmation says changes were requested. A disappears from the Pending queue and appears in Advisor Messages as Changes Requested with the entered feedback. The Student sees the same status/feedback and opens editable A with E1/E2 preselected. **Cancel Plan** is absent for Changes Requested; the instruction to update and resubmit is visible. No approval or rejection has occurred.

UAT-08 — Modify and resubmit the returned plan

Field	Planned specification
Actor / wireframe	Student (Advisor verifies the queue in the named Advisor steps); W2/W5: editable selection, Review Plan, resubmission; W3: revised queue card.
Objective	Apply the requested change and return the revised selection for review.
Preconditions	UAT-07 completed; Student viewing editable A with E1/E2 selected.
Test data	Keep E1; replace E2 with E3; revised total 6 credits.
Procedure	1. In Student **Plan Courses** for A, uncheck MATH 2250 and check ENGL 1000, scrolling the list as necessary. 2. Click **Review Selected Courses** and confirm COMP 1201 plus ENGL 1000 and 6 credits. 3. Click **Resubmit Plan for Advisor Review** and inspect the resulting status and selected-course list. 4. Click Student **Messages** and inspect A's current update. 5. In the Advisor window click **Advisor Review**, refresh, and inspect the Student/A card.
Expected result	A returns to Pending with exactly E1/E3. The old request-changes feedback is cleared on resubmission; Messages reflects the current Pending state rather than promising immutable feedback history. Advisor queue contains one A card for this Student with the revised courses and 6 credits, ready for another decision. Capture the previous feedback in UAT-07 before this step replaces it.

UAT-09 — Approve the revised plan

Field	Planned specification
Actor / wireframe	Advisor (Student verifies delivery in the named Student steps); W3: Comments and Approve; W2/W4: plan status and Messages.
Objective	Approve the revised plan and let the Student see the decision.
Preconditions	UAT-08 completed; revised A Pending.
Test data	A; E1/E3; feedback Revised selection approved.
Procedure	1. In Advisor **Advisor Review**, find the Student/A card and confirm E1/E3. 2. Enter Revised selection approved. into its **Comments (optional)** and click **Approve Plan**. 3. Read the confirmation and refresh the review queue. Click Advisor **Messages** and inspect A. 4. In the Student window click **Messages** and read A's status/feedback. 5. Click **Plan Courses**, choose A, and inspect the selected courses and editing controls. Refresh once.
Expected result	“Plan approved.” appears and A leaves the Pending queue. Both actors see Approved and the exact feedback. Student A retains E1/E3 and remains Approved after refresh. Course checkboxes/review button are locked; Pending-only Remove buttons are absent. **Cancel Plan** remains available because the current implementation permits cancellation of Approved plans; do not cancel until UAT-14.

UAT-10 — Submit a separate plan for rejection

Field	Planned specification
Actor / wireframe	Student; W2: semester, selection, review, submit.
Objective	Submit a different semester without altering the approved plan.
Preconditions	UAT-09 completed; A Approved; B has no plan.
Test data	B = Year 1 semester 2; E2 only, 3 credits.
Procedure	1. Click Student **Plan Courses** and choose **Year 1 semester 2**. 2. Select only MATH 2250; click **Review Selected Courses** and confirm one course, 3 credits. 3. Click **Submit Plan for Advisor Review** and inspect B's status/list. 4. Choose **Year 1 semester 1** to inspect A, then switch back to **Year 1 semester 2**.
Expected result	B is Pending with only E2; A remains Approved with E1/E3. Switching semesters displays the selected semester's own plan and status without mixing the two selections.

UAT-11 — Reject the separate plan

Field	Planned specification
Actor / wireframe	Advisor (Student verifies delivery in the named Student steps); W3: Comments and Reject; W4/W5: rejected status and revision option.
Objective	Reject B and communicate the reason to the Student.
Preconditions	UAT-10 completed; B Pending.
Test data	B; feedback Plan rejected for this UAT scenario; choose a different course mix.
Procedure	1. In the Advisor window click **Advisor Review**, refresh, and find the Student's **Year 1 semester 2** card containing only MATH 2250. 2. Enter the exact feedback above into that card's **Comments (optional)** and click **Reject Plan**. 3. Read the confirmation, refresh the queue, then inspect B in Advisor **Messages**. 4. In the Student window click **Messages** and inspect B's status and feedback. 5. Click B's **Review and update plan** link; inspect the editable course selection and **Cancel Plan** control without using either yet.
Expected result	“Plan rejected.” appears; B leaves the Pending queue. Both actors see Rejected with the exact reason. Student B is editable for resubmission and cancellation is available. A's approval remains unchanged. Leave B Rejected for UAT-15.

UAT-12 — Submit a pending plan for cancellation checks

Field	Planned specification
Actor / wireframe	Student (Advisor verifies the queue in the named Advisor steps); W2: semester, selection, review, submit; W3: queue.
Objective	Prepare a Pending plan through normal browser submission.
Preconditions	UAT-11 completed; C has no plan.
Test data	C = Year 1 semester 3; E1/E2, 6 credits.
Procedure	1. Click Student **Plan Courses**, choose **Year 1 semester 3**, and select COMP 1201 and MATH 2250. 2. Click **Review Selected Courses**, verify 6 credits, and click **Submit Plan for Advisor Review**. 3. Inspect C's Pending status and its selected-course list. 4. In the Advisor window click **Advisor Review**, refresh, and find the Student/C card; make no decision.
Expected result	C is Pending with E1/E2 and is visible to the Advisor with 6 credits. A remains Approved and B remains Rejected. C stays Pending for UAT-13 and UAT-16.

UAT-13 — Remove a course from a Pending plan and protect the last course

Field	Planned specification
Actor / wireframe	Student (Advisor verifies the queue in the named Advisor steps); W2/W5: selected-course removal; submitted-plan Remove is an implementation extension.
Objective	Adjust a submitted Pending plan without leaving an empty plan.
Preconditions	UAT-12 completed; C Pending with E1/E2.
Test data	Remove E2; retain E1, 3 credits.
Procedure	1. In Student **Plan Courses**, choose C and click **Remove** beside MATH 2250 in **Selected courses for Year 1 semester 3**. 2. Dismiss the browser confirmation with **Cancel** and verify both courses remain. 3. Click the same **Remove** again and accept the confirmation with **OK**. 4. Read the confirmation and inspect the remaining course and available removal action. Refresh. 5. Refresh Advisor **Advisor Review** and inspect C's course table and total.
Expected result	Dismissing the dialog changes nothing. Confirming removes only E2 and shows “Course removed from the submitted plan.” C stays Pending with E1 and 3 credits. The remaining course has no Remove button and displays “Cancel the plan to remove its last course.” Advisor queue reflects E1/3 credits after refresh.

UAT-14 — Cancel an Approved plan

Field	Planned specification
Actor / wireframe	Student; W5: cancellation extension to panel 2.
Objective	Cancel A using the implemented permission for Approved plans.
Preconditions	UAT-09 approval retained; A Approved; UAT-12/13 completed.
Test data	A; Approved → Cancelled.
Procedure	1. Click Student **Plan Courses**, choose A, and confirm its current status is Approved. 2. Click **Cancel Plan** and dismiss the confirmation with **Cancel**; verify A remains Approved with E1/E3. 3. Click **Cancel Plan** again and accept the confirmation with **OK**. 4. Read the success message; refresh and inspect status, selected courses, and available actions. 5. Click **Messages** and find A's status.
Expected result	Dismissing cancellation leaves approval unchanged. Confirming shows “Semester plan cancelled. You can edit it and resubmit.” A becomes Cancelled and remains so after refresh; Messages also shows Cancelled. The planning screen clears the active selection, permits a new choice, and hides Cancel Plan. Previous feedback may remain attached to the cancelled record; it must not make the current status appear Approved.

UAT-15 — Cancel a Rejected plan

Field	Planned specification
Actor / wireframe	Student; W5: cancellation extension to returned plan.
Objective	Cancel B instead of resubmitting the rejected selection.
Preconditions	UAT-11 rejection retained; B Rejected.
Test data	B; Rejected → Cancelled.
Procedure	1. Click Student **Plan Courses** and choose B; confirm Rejected and its rejection feedback. 2. Click **Cancel Plan** and accept the confirmation with **OK**. 3. Read the success message, refresh, and inspect the status, selection count, and actions. 4. Click **Messages** and find B's entry.
Expected result	B becomes Cancelled and persists after refresh. No active courses are selected; a new selection is possible and Cancel Plan is absent. Messages reports Cancelled. A's cancellation and C's Pending state are unaffected. Retained rejection feedback does not override the current Cancelled status.

UAT-16 — Cancel a Pending plan and remove it from the review queue

Field	Planned specification
Actor / wireframe	Student (Advisor verifies the queue in the named Advisor steps); W5: Cancel Plan; W3: Pending review queue.
Objective	Withdraw C before the Advisor makes a decision.
Preconditions	UAT-13 completed; C Pending with E1; no Advisor decision on C.
Test data	C; Pending → Cancelled.
Procedure	1. Click Student **Plan Courses**, choose C, and confirm Pending with only COMP 1201. 2. Click **Cancel Plan** and accept the confirmation with **OK**. 3. Read the success message and refresh; inspect the Cancelled banner, selection count, and available actions. 4. Click Student **Messages** and inspect C's status. 5. In the Advisor window click **Advisor Review**, refresh, and look for the Student/C card.
Expected result	C becomes Cancelled, the active selection clears, and no repeat cancellation control appears. Student Messages shows Cancelled. The Student/C card is absent from the refreshed Pending queue, so it cannot be reviewed there. Other students' Pending submissions, if any, remain available.

UAT-17 — Start a new plan after cancellation

Field	Planned specification
Actor / wireframe	Student (Advisor verifies the queue in the named Advisor steps); W2/W5: new selection after cancellation, review and submit; W3: queue.
Objective	Submit a fresh selection for the same semester after withdrawing its previous plan.
Preconditions	UAT-16 completed; C Cancelled with no active selection.
Test data	C; E3 only, 3 credits.
Procedure	1. Click Student **Plan Courses**, choose C, and confirm no course is selected. 2. Select only ENGL 1000 and click **Review Selected Courses**. 3. Confirm ENGL 1000 and 3 credits, then click **Resubmit Plan for Advisor Review** (the implemented label on the cancelled-plan screen). 4. Inspect the resulting Pending status and selected-course list; refresh once. 5. In the Advisor window click **Advisor Review**, refresh, and inspect the Student/C card.
Expected result	C's current plan is Pending with only E3 and 3 credits; the previously cancelled E1 selection does not reappear. Advisor queue contains one active Pending C card for this Student with E3. The screen describes this as a brand new plan even though the action label says Resubmit. Historical Cancelled entries may remain in Messages. Do not infer database identifiers from the browser or require deletion of history.

UAT-18 — Student login and clean-run baseline (execute first)

Field	Planned specification
Actor / wireframe	Student; Student identity/sidebar in panels 1–2; login form is outside the original wireframe
Objective	Verify Student login, identity, empty baseline and persistent session.
Preconditions	Safe local setup complete; fresh database; dedicated Student browser profile signed out; server running
Test data	[Student test account] / [Student test password]; invalid password [invalid Student test password]; semesters A/B/C
Procedure	1. In the Student window open http://127.0.0.1:5002/login. 2. Enter [Student test account] in **Username**, [invalid Student test password] in **Password**, then click **Sign in**. Inspect the response. 3. Enter [Student test account] and [Student test password] and click **Sign in**. 4. Confirm **Student workspace**, Home, Cameron Ramesar and Student ID 12345678/Computer Science. Click **Profile** if needed to read **Username**, **Student ID** and degree program. 5. Click **Home** and confirm the no-submitted-plan message in **Latest semester plan**. Click **Messages** and confirm **No advisor updates yet**. 6. Click **Plan Courses**. Choose **Year 1 semester 1**, then **Year 1 semester 2**, then **Year 1 semester 3**, waiting for each reload. Confirm each has zero selected courses and no existing plan status banner or selected-course list. Return to **Year 1 semester 1**. 7. Click **Home** and refresh. Confirm the Student remains signed in. Continue with UAT-01.
Expected result	Invalid credentials show **Incorrect username or password** on the login page without opening the workspace. Valid credentials open /app as the Student, with the seeded identity, Student navigation and no Advisor Review link. Home, Messages and A/B/C have the fresh empty baseline. Refresh preserves the authenticated Student session. If the baseline is occupied, record a setup blocker and do not begin dependent cases.

UAT-19 — Advisor login (execute after UAT-05, before UAT-06)

Field	Planned specification
Actor / wireframe	Advisor; Advisor identity/navigation in panel 3; shared login form is outside the original wireframe
Objective	Verify Advisor login, identity and isolated persistent session.
Preconditions	UAT-05 complete; A Pending; separate Advisor profile/private session signed out; Student window stays open
Test data	[Advisor test account] / [Advisor test password]; invalid password [invalid Advisor test password]
Procedure	1. In the isolated Advisor window open http://127.0.0.1:5002/login. 2. Enter [Advisor test account] in **Username**, [invalid Advisor test password] in **Password**, and click **Sign in**. 3. Check the login error, then enter [Advisor test account] / [Advisor test password] and click **Sign in**. 4. Inspect **Advisor workspace**, Home, Dr. Williams and the **Advisor Review** sidebar link. Click **Profile** and check Username supplied for the Advisor test account and Role **Advisor administrator**. 5. Click **Home** and refresh. Confirm the Advisor session remains active. Refresh Student **Home** in its separate window and confirm it still shows the Student identity. Continue with UAT-06 in the Advisor window.
Expected result	Invalid credentials show **Incorrect username or password**. Valid login opens Advisor Home at /app, with Advisor navigation and no Degree Progress or Plan Courses sidebar links. It does not automatically open the review queue. Refresh retains the Advisor session, and the separate Student session is unaffected. If Student identity changes, stop and correct profile isolation before making decisions.

UAT-20 — Browser-visible role and signed-out restrictions (execute last)

Field	Planned specification
Actor / wireframe	Student (Advisor checks use the isolated Advisor window in steps 4–6); Different Student navigation in panels 1–2 and Advisor navigation in panel 3; access-denied pages are supporting implementation
Objective	Verify browser role restrictions and signed-out access denial.
Preconditions	UAT-17 complete; both role sessions authenticated and isolated; A/B Cancelled, new C Pending with E3
Test data	Direct URLs /admin, /app?tab=plan-courses and /app?tab=degree-progress; no plan IDs or request tampering
Procedure	1. In Student **Home**, confirm **Degree Progress** and **Plan Courses** links exist and **Advisor Review** is absent. 2. In that Student window type http://127.0.0.1:5002/admin directly into the address bar and load it. 3. Record the response. Return by typing http://127.0.0.1:5002/app?tab=home; confirm Student Home still opens. 4. In the Advisor window confirm **Advisor Review** exists while **Degree Progress** and **Plan Courses** sidebar links are absent. 5. In the Advisor address bar load http://127.0.0.1:5002/app?tab=plan-courses, then http://127.0.0.1:5002/app?tab=degree-progress. Inspect the content after each load. 6. Click **Advisor Review** and verify the Student/C Pending card still contains only ENGL 1000, 3 credits. Make no decision. 7. In the Student window click **Sign out**. Type http://127.0.0.1:5002/app in the address bar and load it afresh (do not rely on a cached Back page). Inspect the response, then click **Return to Login**. 8. In the Advisor window click **Sign out**, then directly load http://127.0.0.1:5002/admin. Inspect the response and click **Return to Login**.
Expected result	Student direct access to /admin is denied with **You are not authorized to access this resource** (HTTP 403; the browser can display JSON, not a custom denial dashboard). No queue or review controls are exposed. Advisor attempts to use Student-only tab query values render **Advisor Home** at /app, because unsupported tabs fall back to Home; they do not display a Student planning/progress screen and do not promise HTTP 403. The valid Advisor queue remains accessible with unchanged C. After sign-out, direct /app and /admin loads show the implemented **401 Unauthorized / Session Expired** page with **Return to Login**. No plan state changes occur. This case demonstrates these specific browser restrictions only. It does not test forged POSTs, all API permissions, or another Student's ownership. Do not mark those as manually passed by inference.

6.2 Unit Test Plan

Objective and boundary. Verify business decisions in isolation from HTTP, templates and persistence. The tests exercise the real AdvisingService with autospecced repository doubles and in-memory SQLModel objects, and reject accidental database connections. AuthService orchestration uses mocked repository/security collaborators. UT-23 exercises real local cryptography with controlled time and test-only settings, without database or network access.

Components. AdvisingService dashboard totals, prerequisite eligibility, semester and selection validation, submission delegation, plan edit/review/cancel/removal rules; AuthService; current-user/role dependencies; password and JWT utilities. UT-01–23 identify specifications, not counts of parameterized executions.

Fixture / design	Planned specification
Actors / terms	S1 Student id 1; S2 independent Student; A1 Advisor. T1 = Year 1 semester 1; T2 = Year 1 semester 2.

Fixture / design	Planned specification
Courses	C1 id101 COMP 1100, Core; C2 id102 MATH 1000, General Education; C3 id103 COMP 2000, Core, requires C1; C4 id104 MATH 2000, Electives, no prerequisite; C5 id105 COMP 3000, requires C1 and unknown MATH 9999. Each is 3 credits.
Default state	C1 Completed; C2 In Progress; C3/C4 untracked; 93-credit degree; Core requirement 36; empty plan history. Cases override explicitly. F1 is synthetic test data, distinct from the browser seed.
Positive / negative / boundary	Positive: eligible selections and legal decisions. Negative: invalid terms, duplicates, missing/foreign resources, bad identity. Boundary: zero totals, cap at 100%, duplicate records, last remaining course and 0/499/500/501 raw feedback characters.
Oracle	Literal independently expected values and exact repository calls/no calls. The method under test is never mocked and the expected answer is not derived by calling it. Observed outcomes are not included in these plan tables.

Contract note. UT-22 is adapted to the current 401/403 role contract. E01/E06/E07/E08 references denote existing database-backed baseline tests in tests/test_advising_service.py; their planning notes avoid duplicate coverage.

Case / planned inputs and setup	Expected output (not an observed result)
UT-01 — Degree totals and category calculation **Component:** AdvisingService.get_student_dashboard **Preconditions:** Mock repositories return fixture S1 and no plans. **Inputs:** F1 baseline: completed C1=3; in-progress C2=3; program=93; Core requirement=30. **Procedure:** Call dashboard(S1,T1); inspect totals, percentage and category output.	Completed=3; in-progress=3; remaining=87; total=93; progress=3%; Core earned=3, remaining=27, percent=10. E01 already checks baseline persistence: add only isolated calculation assertions.
UT-02 — Empty academic record and missing program fallback **Component:** AdvisingService.get_student_dashboard **Preconditions:** Mock user/program/record lookups; no plans. **Inputs:** Variants: existing 93-credit program with no records; missing degree program. **Procedure:** Call each variant with empty completed/in-progress records.	Credits=0/0, remaining=93, progress=0; category earned=0 for configured requirements; absent program uses 93 and category_progress=[].
UT-03 — Progress limits, zero denominators and duplicate records **Component:** AdvisingService.get_student_dashboard **Preconditions:** Repository doubles only; deliberately synthetic boundary data. **Inputs:** Variants: total=6, completed=9, in-progress=3; total=0; category required=0; repeated completed record for C1. **Procedure:** Call each variant; compare arithmetic independently.	Remaining never negative; progress and category percent capped at 100; zero denominator yields 0; duplicate same-status course records count its credits once. Do not claim service sanitizes all inconsistent records.
UT-04 — Prerequisites satisfied or absent **Component:** AdvisingService.prerequisite_ok; get_course_availability **Preconditions:** Mock catalogue, completed IDs and course records. **Inputs:** C3 requires C1; C4 requires none; advanced course requires ' COMP 1100, MATH 1000 '; both completed in last variant. **Procedure:** Call both functions for each target; compare bool and availability message.	Untracked targets eligible if all named prerequisites completed; whitespace ignored; no prerequisites eligible; availability=(True, 'Eligible').

Case / planned inputs and setup	Expected output (not an observed result)
UT-05 — Missing and in-progress prerequisites **Component:** AdvisingService.prerequisite_ok; get_course_availability **Preconditions:** Mock completed IDs exclude C2; C2 is In Progress. **Inputs:** C3 requiring C2; advanced course requiring C1 and unknown MATH 9999; blank comma-separated tokens. **Procedure:** Evaluate both eligibility functions per variant.	In-progress prerequisite does not satisfy completion; unknown code fails; availability names only unmet codes; empty comma tokens ignored. E08 supplies DB-backed missing-code baseline.
UT-06 — Tracked or unknown target courses **Component:** AdvisingService.prerequisite_ok; get_course_availability **Preconditions:** Mock repository returns completed C1, in-progress C2 and absent ID 999999. **Inputs:** Targets C1, C2, 999999. **Procedure:** Evaluate prerequisite_ok for all; inspect catalogue availability for C1/C2.	False for all three; C1 says Already completed; C2 says Already in progress; nonexistent course absent from availability map.
UT-07 — Valid semester names and trimming **Component:** AdvisingService.submit_plan; SEMESTER_OPTIONS **Preconditions:** Mock eligible C3, no existing plan, create_plan sentinel. **Inputs:** All nine Year {1..3} semester {1..3}; padded ' Year 1 semester 1 '. **Procedure:** Parameterize valid values; submit [C3]; inspect create_plan arguments.	Each accepted; exactly nine options including semester 3; padded input stored trimmed. E06 baseline reused rather than copied.
UT-08 — Invalid semester names **Component:** AdvisingService.submit_plan **Preconditions:** Mock repositories; mutation methods are spies. **Inputs:** '', whitespace, Fall 2026, Year 0 semester 1, Year 4 semester 1, Year 1 semester 4, Year 1 Semester 1. **Procedure:** Submit [C3] for each input; capture exception and calls.	ValueError 'Choose a valid year and semester before submitting a plan.'; no create/resubmit call. E07 reused for its legacy-name variant.
UT-09 — Empty selection **Component:** AdvisingService.submit_plan **Preconditions:** No database; repository spies. **Inputs:** [] with valid T1. **Procedure:** Call submit_plan(S1,[],T1).	ValueError 'Select at least one course for the semester plan.'; no persistence or eligibility lookup.
UT-10 — Duplicate selected course IDs **Component:** AdvisingService.submit_plan **Preconditions:** No database; repository spies. **Inputs:** [C3,C3] and [C3,C4,C3] with T1. **Procedure:** Submit each duplicate list.	ValueError 'A course may only appear once in a semester plan.'; no create/resubmit call. Distinct from duplicate semester plan in E05.
UT-11 — Missing or ineligible selection aborts submission **Component:** AdvisingService.submit_plan **Preconditions:** Mock availability and course lookup, no real repository operations. **Inputs:** [C3,999999]; [C1]; [C2]; [advanced course missing MATH 9999]. **Procedure:** Submit each selection; inspect all mutation spies.	Missing ID reports no longer available; ineligible selection includes course code and reason; no create/resubmit call even when earlier list item was valid.
UT-12 — Submission status decision table **Component:** AdvisingService.submit_plan **Preconditions:** Mock existing-plan lookup and mutation return sentinels. **Inputs:** Existing status: none, Pending, Approved, Rejected, Changes Requested, Cancelled; valid [C3],T1. **Procedure:** Submit each variant separately and assert exactly which repository method is called.	None/Cancelled -> create_plan; Rejected/Changes Requested -> resubmit_plan(existing,[C3]); Pending/Approved -> ValueError already under review or approved and no mutation. Database replacement/reset details belong to integration tests.

Case / planned inputs and setup	Expected output (not an observed result)
UT-13 — Dashboard edit and cancelled-selection rules **Component:** AdvisingService.get_student_dashboard **Preconditions:** Mock current plan and its C3 link, no DB. **Inputs:** Each status Pending, Approved, Rejected, Changes Requested, Cancelled; also no plan. **Procedure:** Build dashboard per variant; inspect selected IDs, planned courses, can_edit_plan.	Edit true only Rejected/Changes Requested/Cancelled; false Pending/Approved/no plan. Cancelled hides saved course links; other existing statuses retain selected C3.
UT-14 — Advisor decisions and feedback normalization **Component:** AdvisingService.review_plan **Preconditions:** Mock Pending plan, update_plan sentinel. **Inputs:** approve, reject, request_changes; feedback=' Review note '; advisor=A1. **Procedure:** Call each decision separately; inspect update_plan keywords.	Mapped statuses Approved, Rejected, Changes Requested; feedback='Review note'; advisor_id=A1; exactly one update call. Empty feedback is allowed for every decision.
UT-15 — Invalid advisor decision **Component:** AdvisingService.review_plan **Preconditions:** Repository lookup/update spies. **Inputs:** '', APPROVE, cancel, unknown; valid short feedback. **Procedure:** Call review_plan per decision.	ValueError 'Choose approve, reject, or request changes.' before persistence; no update call.
UT-16 — Feedback length boundary before trimming **Component:** AdvisingService.review_plan **Preconditions:** Mock Pending plan and update spy. **Inputs:** For each legal decision: lengths 0,499,500,501; 501 spaces; ' '+499 characters+' ' (501 total). **Procedure:** Call review_plan per variant; compare raw and stripped lengths.	0/499/500 accepted, stored stripped; 501 rejected with 'Advisor feedback must be 500 characters or fewer.' even if trimming would shorten it; no update on rejection.
UT-17 — Review only existing Pending plans **Component:** AdvisingService.review_plan **Preconditions:** Mock lookup absent or returning status variant. **Inputs:** Missing plan; Approved, Rejected, Changes Requested, Cancelled; valid approve/feedback. **Procedure:** Call review_plan for each variant.	Missing -> ValueError 'Plan not found'; each non-Pending -> 'Only pending plans can be reviewed.'; no update.
UT-18 — Cancellation status and ownership matrix **Component:** AdvisingService.cancel_plan **Preconditions:** Mock owned/foreign/missing plan and cancel spy. **Inputs:** Owned Pending, Approved, Rejected, Changes Requested, Cancelled; foreign S2 plan; missing plan. **Procedure:** Cancel as S1 per independent variant.	Only owned Pending/Approved/Rejected delegated to repo.cancel_plan; other owned statuses reject; foreign/missing -> 'Plan not found.'; no mutation on denial.
UT-19 — Remove course guard matrix **Component:** AdvisingService.remove_submitted_course **Preconditions:** Mock owned Pending plan with C3/C4 links and remove spy. **Inputs:** Remove C4 from two links; last C3; absent course; non-Pending statuses; foreign or absent plan. **Procedure:** Call each variant independently; inspect remove_plan_course call.	Only an owned Pending plan with matching course and more than one link permits deletion; others reject with existing guard messages; no deletion on failure.
UT-20 — Authentication success **Component:** AuthService.authenticate_user **Preconditions:** Mock UserRepository; patch verify_password and create_access_token in app.services.auth_service. **Inputs:** Stored user id=S1, role=regular_user, password='stored-hash'; verifier=True. **Procedure:** Authenticate with username and plaintext; inspect calls.	Returns token sentinel; verifier receives plaintext and stored hash; token data={'sub': str(S1), 'role':'regular_user'}; no real DB/hash/JWT needed.
UT-21 — Authentication failures and short circuit **Component:** AuthService.authenticate_user **Preconditions:** Mock repository and security collaborators. **Inputs:** Unknown username; known user with verifier=False. **Procedure:** Authenticate each case; inspect verifier/token calls.	Returns None; missing user never invokes verifier; neither case issues token. No token or password material recorded as evidence.

Case / planned inputs and setup	Expected output (not an observed result)
UT-22 — Cookie token resolution and admin role **Component:** get_current_user; is_admin_dep in app.dependencies.auth **Preconditions:** Synthetic Request, mocked jwt.decode/settings/UserRepository; async invocation. **Inputs:** Missing cookie; decoder InvalidTokenError; nonexistent subject user; valid S1; valid A1. **Procedure:** Call current-user resolver per variant; call admin guard with resolved S1 and A1.	Missing/invalid/unresolved identity raises HTTP 401; valid identity resolves the repository user. The current require_roles guard denies regular_user at the Advisor guard with 403 and returns admin. Test the current database role independently of a token role claim.
UT-23 — Password hash and JWT utility contracts **Component:** encrypt_password; verify_password; create_access_token **Preconditions:** No DB; actual local hash/JWT utilities, fixed test settings and controlled clock. **Inputs:** Test-only password; correct/wrong password; claims sub=S1,role=regular_user; explicit 5-minute lifetime. **Procedure:** Hash/verify locally; create token at fixed UTC time and decode with test key; compare input claims after call.	Hash differs from plaintext; correct verifies true, wrong false; signed token carries sub/role and exp at fixed time +5 minutes; input dict not mutated. Patch time/settings only, not functions under test.

6.3 Integration Test Plan

Objective and boundary. Join real FastAPI routes and dependencies, form/JSON parsing, real cookie/JWT authentication, services, repositories and SQLModel persistence. Exercise Student and Advisor workflows and database rollback. IT-19/20 intentionally reach database failures directly because service validation rejects those inputs before SQL.

Setup / coverage	Planned specification
F2 data	Reuse F1 course/degree meanings but resolve generated row IDs. Seed two Students S1/S2 and Advisor A1, real password hashes, C1 Completed/C2 In Progress for S1, distinctive S2-only records/feedback, and independent plan state per variant.
Isolation	Fresh per-case SQLite engine (StaticPool, check_same_thread=False), metadata and foreign keys enabled. New Session per HTTP request and per persistence check. Separate Student/Advisor cookie jars; follow_redirects=False.
Startup and cleanup	Rebind test engine/session dependencies and startup/recovery initialization so normal/private databases cannot be touched. Do not replace authentication, handlers, business methods or CRUD. Close clients/sessions, restore overrides/lifespan/settings and dispose the engine.
Coverage	Real login/logout; invalid credentials/tokens; current database roles; course eligibility; Student submission and persistence; Advisor queue and decisions; feedback/resubmission; removal/cancellation/replacement; ownership/privacy; invalid schemas, constraints and atomic rollback.
Current contracts	Unauthenticated protected HTML/JSON: 401. Authenticated wrong-role guard: 403. HTML business errors: 303 plus a warning flash. Strict JSON APIs use documented 401/403/404/409/422 semantics.

Contract note. IT-01/12/13 use the current 401/403 contract in place of the historical H3 wording. The original source documents remain unchanged; these are planned expectations, not new observations.

Case / planned input and procedure	Expected output (not an observed result)
IT-01 — Real login, cookie and logout **Component:** POST /login; GET /app; GET /logout **Preconditions:** F2 database, separate unauthenticated client, real hash/JWT/auth dependencies. **Inputs:** S1 correct password; wrong password; unknown username. **Procedure:** POST form credentials for each variant; capture cookie/redirect; with successful client GET /app; GET /logout then protected GET.	Success ->303 /app and access_token cookie, authenticated page 200; wrong/unknown ->303 /login, Incorrect username or password flash, no new access_token on clean client; logout ->303 /login, cookie removed, protected HTML/JSON access denied with HTTP 401.

Case / planned input and procedure	Expected output (not an observed result)
IT-02 — Student degree progress through HTTP **Component:** GET /app?tab=degree-progress&semester=T1 **Preconditions:** F2; login S1 using IT-01 setup. **Inputs:** C1 Completed=3; C2 In Progress=3; 93-credit program; Core required=30; S2 distinct records. **Procedure:** GET page; inspect rendered totals and category; query records in fresh session.	200; completed=3, in-progress=3, remaining=87, overall=3%; Core earned=3/30 and remaining=27; no S2 activity or profile; no database mutation.
IT-03 — Eligible selection, submission and advisor queue **Component:** GET /app?tab=plan-courses; POST /app/plan; GET /admin **Preconditions:** F2; S1 and A1 authenticated in separate clients; no T1 plan. **Inputs:** Repeated form course_ids=C3 and C4; semester_name=T1. **Procedure:** GET catalogue; inspect eligible/disabled options; POST form selection; follow redirect manually; GET /admin as A1; query saved plan/links.	Catalogue eligibility reflects completed C1 and in-progress C2; POST ->303 with semester and tab=plan-courses; success flash; exactly one Pending S1/T1 plan, two unique links, submitted_at set, advisor_id=None; appears in real advisor queue.
IT-04 — Persistence across requests and sessions **Component:** POST /app/plan; GET /app; AdvisingRepository.get_plan_for_student/get_plan_courses **Preconditions:** F2; S1 authenticated; no T1 plan. **Inputs:** [C3,C4],T1. **Procedure:** Submit through HTTP; close request-scoped session; open a fresh Session on same isolated engine; read plan/links; issue another GET.	Committed plan ID, S1, T1, Pending and both links survive session boundaries and render as selected; no reliance on Python object identity. E02/E09 do not prove full authenticated request chain.
IT-05 — Advisor approval and rejection **Component:** POST /admin/review/{plan_id}; GET /admin; GET /app?tab=messages **Preconditions:** F2; seed a separate Pending S1 plan for each decision; S1 and A1 logged in. **Inputs:** approve with ''; reject with 'Please choose another course.'; JSON and form variants. **Procedure:** A1 POST each review; inspect redirect/flash; read row in fresh session; GET advisor queue and student messages.	303 /admin; matching success flash; Approved or Rejected stored with A1 and stripped feedback; reviewed plan leaves Pending queue; student sees own decision/message; plan course links preserved.
IT-06 — Request changes and resubmit **Component:** POST /admin/review/{id}; POST /app/plan; GET /app and /admin **Preconditions:** F2; Pending S1/T1 plan containing C3, A1/S1 authenticated. **Inputs:** request_changes, feedback=' Choose another course. '; replacement=[C4]. **Procedure:** A1 request changes through HTTP; verify persisted status/student message; S1 submit replacement for same semester; query fresh session and advisor queue.	Changes Requested, A1, trimmed feedback initially; resubmission ->303; same plan ID becomes Pending; links exactly [C4]; old C3 link removed; advisor_id=None, feedback=''; submitted_at refreshed; returns to Pending queue.
IT-07 — Rejected plan resubmission without cancellation **Component:** POST /admin/review/{id}; POST /app/plan **Preconditions:** F2; S1/A1 authenticated; S1 Pending T1 plan containing C3. **Inputs:** reject with feedback; replacement=[C4]. **Procedure:** Reject via HTTP; submit C4 for same semester without cancel; read fresh-session plan and links.	303 for both requests; same ID reset to Pending; C4 replaces C3; feedback and advisor cleared; timestamp refreshed. Distinct from E10 cancellation/new-ID behavior.
IT-08 — Allowed cancellation and replacement **Component:** POST /app/plan/{id}/cancel; POST /app/plan; GET /admin and /app **Preconditions:** F2; independently owned Pending, Approved and Rejected plans; S1 logged in. **Inputs:** Each allowed status with existing [C3] links; replacement [C4], same semester. **Procedure:** Cancel per variant; inspect row, links, UI and advisor queue; submit replacement; query all matching plan rows.	303; original row Cancelled and retains links/history; hidden from Pending queue; cancelled plan displays no selected/planned courses; replacement has new ID and Pending status, old Cancelled row remains.

Case / planned input and procedure	Expected output (not an observed result)
IT-09 — Forbidden cancellation statuses **Component:** POST /app/plan/{id}/cancel **Preconditions:** F2; S1 authenticated; independently owned Changes Requested and Cancelled plans. **Inputs:** Each disallowed status; unchanged links/feedback snapshots. **Procedure:** POST cancel per variant; follow redirect for warning; query fresh state.	303 tab=plan-courses with 'Only pending, rejected, or approved plans can be cancelled.' warning; status, links and feedback unchanged. Changes Requested permits resubmission, not cancellation.
IT-10 — Submitted course removal and last-course rule **Component:** POST /app/plan/{id}/courses/{course_id}/remove **Preconditions:** F2; S1 authenticated, owned Pending plan [C3,C4]; separate non-Pending variants. **Inputs:** C4 then C3; absent course ID; each Approved/Rejected/Changes Requested/Cancelled. **Procedure:** Remove C4; try last C3; test absent and non-Pending variants independently; query DB after each.	First request 303/success, one C3 link remains; last-course, absent-course and non-Pending requests 303/warning without state change; no orphaned/deleted plan.
IT-11 — Cross-student ownership and data visibility **Component:** GET /app?tab=degree-progress; GET /app?tab=messages; POST /app/plan/{id}/cancel; POST /app/plan/{id}/courses/{course_id}/remove; POST /app/plan **Preconditions:** F2; S1 and S2 logged in separately; S2 owns Pending [C3,C4] and distinctive records/feedback. **Inputs:** S1 uses S2 plan ID for cancel/remove; submits valid own selection with extra student_id=S2 field. **Procedure:** S1 GET dashboard/messages; POST foreign cancel/remove; POST own plan with forged student_id field; compare DB before/after.	Foreign operations ->303 warning 'Plan not found.' with S2 row/links untouched; no S2 private record/message in S1 output; new submission belongs to cookie-authenticated S1, extra form student_id does not override identity.
IT-12 — Unauthenticated and invalid-token access **Component:** GET /app; GET /admin; POST /app/plan; POST /admin/review/{id}; POST /app/plan/{id}/cancel; POST /app/plan/{id}/courses/{course_id}/remove **Preconditions:** F2; valid target plan; clean client per token variant; no auth override. **Inputs:** No cookie; malformed token; expired correctly signed token; wrong-signature token; valid token for nonexistent user. **Procedure:** Send each protected request with otherwise valid payload under each variant; compare entire relevant DB state.	Protected HTML and JSON requests return HTTP 401 for missing, malformed, expired, wrong-signature or nonexistent-user tokens, with no private data and no database writes.
IT-13 — Student cannot use advisor operations **Component:** GET /admin; POST /admin/review/{id} **Preconditions:** F2; S1 logged in; Pending S2 plan. **Inputs:** Valid approve/reject/request_changes payloads as regular_user. **Procedure:** GET queue; POST each decision; query plan in fresh session.	Authenticated Student access to the Advisor queue and review operations returns HTTP 403. No queue/private data or status/feedback/advisor/course-link changes.
IT-14 — Business-invalid submission inputs **Component:** POST /app/plan **Preconditions:** F2; S1 logged in; no plan for target semester; reset DB per variant. **Inputs:** Omitted/empty course_ids; [C3,C3]; [999999]; [C1] completed; [C2] in progress; unmet prerequisite C5; invalid/blank semester; valid C3 plus invalid item. **Procedure:** POST form per variant; capture redirect; GET warning; compare SemesterPlan/PlanCourse counts.	303 warning for each business rejection, no partial plan/links. Messages identify empty/duplicate/missing/ineligible selections or invalid semester. Whitespace around a valid semester is accepted and stored trimmed (positive control).
IT-15 — Duplicate active semester plan **Component:** POST /app/plan **Preconditions:** F2; S1 logged in; owned T1 plan separately Pending or Approved. **Inputs:** Same-semester [C4]; second positive control uses T2. **Procedure:** POST duplicate; snapshot row/links; then submit to T2.	Pending/Approved duplicate ->303 warning already under review or approved; original plan/links unchanged, no extra T1 plan; independent T2 submission accepted.

Case / planned input and procedure	Expected output (not an observed result)
IT-16 — Invalid review payload shapes and syntax **Component:** POST /admin/review/{id} **Preconditions:** F2; A1 logged in, Pending plan; reset per variant. **Inputs:** Malformed JSON '{'; JSON [] or null; {}; decision='' or integer; feedback=null/list/integer; noninteger plan path ID; valid form control. **Procedure:** POST with accurate Content-Type per variant; capture raw response; compare persisted snapshot.	Malformed JSON ->400; non-object/missing-or-invalid decision/non-string feedback/noninteger path ->422; no state changes; valid form decision with string feedback ->303 and normal review. JSON syntax checks must send raw malformed bytes, not client JSON serialization.
IT-17 — Review business validation and feedback boundary **Component:** POST /admin/review/{id} **Preconditions:** F2; A1 authenticated; independent Pending/non-Pending/missing plan cases. **Inputs:** Unknown decision; feedback lengths 0,499,500,501; 501 spaces; non-Pending statuses; missing integer ID. **Procedure:** POST JSON and form boundary variants; follow redirect for flash; query fresh state.	Business-invalid cases ->303 /admin warning, unchanged DB; valid 0/499/500 feedback accepted and trimmed; 501 rejected before trimming. Missing plan says Plan not found; non-Pending says Only pending plans can be reviewed. No invented mandatory-feedback rule.
IT-18 — Invalid form types and path identifiers **Component:** POST /app/plan; POST /app/plan/{id}/cancel; POST /app/plan/{id}/courses/{course_id}/remove **Preconditions:** F2; S1 logged in, owned Pending [C3,C4]. **Inputs:** course_ids='abc' or mixed [C3,'abc']; plan_id='abc'; course_id='abc'; missing integer plan ID 999999; missing course membership. **Procedure:** Send repeated form fields or path variants; capture validation response and DB snapshot.	Noninteger form/path IDs ->422 before service mutation; nonexistent integer plan or absent plan course ->303 warning; plan and links unchanged. Student submission endpoint expects form data; do not assume a JSON REST contract.
IT-19 — Database uniqueness constraints **Component:** PlanCourse and StudentCourse SQLModel tables; AdvisingRepository.create_plan **Preconditions:** F2 fresh engine with foreign keys enabled; seed referenced users/courses and valid Pending plan. **Inputs:** Duplicate (plan_id,C3); duplicate (S1,C1); invalid foreign-key link. **Procedure:** Use isolated repository/session boundary to insert each invalid row; catch IntegrityError; rollback; open fresh session.	Real database rejects duplicate pairs and dangling foreign key; committed valid rows remain intact. This is a DB integration case rather than HTTP; UT-10 covers the earlier service guard.
IT-20 — Create/resubmit transaction rollback **Component:** AdvisingRepository.create_plan; resubmit_plan **Preconditions:** F2 real isolated DB; existing Rejected plan [C3] for resubmit case. **Inputs:** Create with [C4,C4]; resubmit existing plan with [C4,C4] to trigger real unique-constraint failure. **Procedure:** Invoke repository directly to reach DB failure past service duplicate guard; catch IntegrityError; inspect a fresh session after rollback.	Failed create leaves no new plan or links; failed resubmit preserves original status/feedback/advisor/timestamp/C3 links; session usable after rollback. Keep actual DB/CRUD real; do not mock commit. Normal route persistence remains covered by IT-03/04/06.

7. Implementation

7.1 GitHub Repository

Repository: https://github.com/Camcoon/MyAdvisorWithTests. Default branch **main**. The published repository was verified on 4 November 2026 at commit 847e39a9cc6340e51bcc05c9864f75f8457021c1; the repository, tree and required file links returned HTTP 200.

Artifact	Published location	Verification
Application source	app/	Current FastAPI application, models, services, repositories and routers are present.
93-credit configuration	app/demo_requirements.py	Core 36; General Education 18; Electives 12; Major 18; Additional 9; total 93.
Unit tests	tests/unit/	test_advising.py and test_auth.py are present.
Integration tests	tests/integration/	Student, Advisor, database-integrity, audit-correction and contrast test files are present.
Postman collection	docs/assignment2/postman/	MyAdvisor-Assignment2.postman_collection.json v2.1.0; 90 requests; revision 2026.10.27-degree93.
UAT screenshots	docs/assignment2/screenshots/UAT/	All 55 case-level captures referenced by the execution record are present.
Test plans	docs/assignment2/test-plans/	UAT, unit and integration plans are present.
Reports	docs/assignment2/reports/	UAT, unit, integration and Postman reports are present.
Project documentation	README.md; docs/report.md; docs/diagrams/; docs/wireframes/	Project documentation, use case diagram and wireframe are present.

The published repository is the source of record for the application, tests, plans, reports, screenshots and exported Postman JSON. The 93-credit values are fictional coursework demonstration data, not official UWI curriculum data.

7.2 Unit Tests

The unit suite is selected with -m unit. tests/unit/conftest.py patches repository constructors where AdvisingService imports them, supplies autospecced doubles and in-memory model objects, and guards database connections. The real service logic is preserved and mutation calls are tracked. tests/unit/test_advising.py implements UT-01–19; tests/unit/test_auth.py implements UT-20–23.

Functions under test include get_student_dashboard, prerequisite_ok, get_course_availability, submit_plan, review_plan, cancel_plan, remove_submitted_course, authenticate_user, get_current_user, the role guard and the password/JWT utilities. Parameterization covers legal and illegal status matrices and raw feedback-length boundaries. UT-23 uses real hashing and signing; credentials and tokens are not printed.

Actual unit implementation example. The following is a verbatim test from tests/unit/test_advising.py; it asserts both the empty-selection error and the early short-circuit behaviour.

```
def test_UT_09_empty_selection(advising):
    with pytest.raises(ValueError, match="^Select at least one course for the semester plan\\.$"):
        advising.service.submit_plan(1, [], T1)
    no_submission(advising)
    advising.repo.list_courses.assert_not_called()
    advising.repo.get_completed_course_ids.assert_not_called()
    advising.repo.get_completed_courses.assert_not_called()
```

Source: tests/unit/test_advising.py; tests/unit/test_auth.py.

7.3 Integration Tests

FastAPI and Starlette TestClient send real requests through the application. tests/integration/conftest.py creates a fresh SQLite engine and schema per case, enables foreign keys, isolates startup/recovery, overrides request sessions and opens independent actor clients. Real password hashing, JWT decoding, current database role lookup, request parsing, services and CRUD remain active. Teardown restores application settings and overrides and closes database resources.

Test file	Actual implementation
tests/integration/test_student_workflow.py	Student login/cookies/logout, progress/catalogue, form and JSON submission, committed state, ownership/privacy, validation, removal, cancellation and same-ID resubmission.
tests/integration/test_advisor_workflow.py	Real Student-to-Advisor queue delivery; decisions through form, generic JSON and named JSON actions; feedback; resubmission and replacement; authorization and validation boundaries.
tests/integration/test_database_integrity.py	Real uniqueness and foreign-key constraints and rollback for failed create/resubmit; fresh-session checks of committed state.
tests/integration/test_audit_corrections.py	Additional role-denial, malformed-email, registration and application-engine integrity variants.
tests/integration/test_contrast_regression.py	Rendered-template contrast variants for normal, hover and Approved states.

Actual workflow implementation example. The following is a verbatim opening of the cancellation, queue and replacement test. A real Student cancels a Pending plan, a fresh database snapshot verifies the Cancelled status and retained links, and the real Advisor queue becomes empty. The remainder submits a replacement and verifies the new Pending plan.

```
def test_cancelled_plan_leaves_advisor_queue_and_replacement_returns(student):
    env, client = student.env, student.client
    advisor = env.login("a1")
    plan_id = env.seed_plan()
    queue(advisor, [plan_id])
    before = env.snapshot()
    response = client.post(f"/app/plan/{plan_id}/cancel")
    assert response.status_code == 303
    assert "Semester plan cancelled." in client.get(response.headers["location"]).text
    assert env.plan(plan_id) == {**before["semesterplan"][0], "status": "Cancelled"}
    assert env.snapshot()["plancourse"] == before["plancourse"]
    queue(advisor, [])
```

Source: student workflow; advisor workflow; database integrity.

7.4 API Tests — Postman

Collection: MyAdvisor - COMP 3613 Assignment 2. **Format:** Postman Collection v2.1.0. **Repository path:** docs/assignment2/postman/MyAdvisor-Assignment2.postman_collection.json. **Published JSON:** docs/assignment2/postman/MyAdvisor-Assignment2.postman_collection.json. The link was verified on 4 November 2026 and returned HTTP 200.

The collection contains **90 ordered requests** in five folders. It covers all 15 implemented JSON API operations and supporting health, login, registration and logout calls. The published revision is 2026.10.27-degree93 and uses the approved 93-credit allocation. Request 01 initializes run variables; later requests depend on earlier state, so the collection is executed once in the original 01–90 order.

Ordered request group	Requests	Coverage
01 Readiness and real cookie authentication	01–12 (12)	Health; anonymous access; malformed cookie; bad/valid Student and Advisor login; independent Student registration/login.

Ordered request group	Requests	Coverage
02 Role boundaries, progress and catalogue	13–21 (9)	Role denials; response privacy; progress; dynamic eligible/blocked course discovery; free-semester selection.
03 Student plan lifecycle and validation	22–50 (29)	Empty, duplicate, unknown, ineligible and invalid submissions; persistence; foreign ownership; duplicate active plan; Pending removal and last-course protection.
04 Advisor decisions and resubmission	51–76 (26)	Queue/detail; invalid decisions and shapes; 500/501-character feedback; change request; same-ID resubmission; reject and approve.
05 Cancellation, history and logout	77–90 (14)	Preserved approval; cancellation/history/new-ID replacement; queue cleanup; actor logout and no-cookie denial.

Authentication. Login requests capture each actor's access_token into separate collection variables. Requests send the intended actor's Cookie explicitly; the prescribed run disables automatic cookie-jar and redirect handling. Exported password and token values are blank. Run-only credentials must be supplied privately on a disposable local instance.

Positive and negative testing. Valid operations are paired with expected 401/403/404/409/422 denials, malformed JSON and path IDs, spoofed fields, foreign resources, illegal status transitions and feedback boundaries. Read-back checkpoints verify that invalid or forbidden operations do not change the protected plan. Dynamic variables carry the base URL, actor tokens, discovered course IDs, selected semester and generated plan IDs.

Workflow coverage. WF1 uses request 19 for degree calculations and privacy. WF2 uses catalogue and eligibility (20), semester choice and invalid input (21–30), persisted submission (31–32), duplicate/state/ownership guards (33–47), removal and last-course checks (48–50), revision and returned status (63–72), and cancellation/history/replacement (78–86). WF3 covers queue/detail (51–53), malformed or invalid review/feedback/identity (54–60), requests for changes and feedback (61–68), rejection at the 500-character boundary (69–72), approval and state guards (73–77).

The historical Runner result in Section 8.3 (90 requests, 470 assertions, zero failures, zero execution errors) belongs to the earlier exported collection revision. The published degree93 revision passed static validation but has not been rerun in Postman. No new Runner pass is claimed.

8. Reports

The results below are observed executions and are kept separate from the planned outcomes in Section 6. No pytest, Postman or browser UAT was rerun to prepare this document.

Execution	Verified result	Build / evidence qualification
Historical unit-marker run	113 passed; 602 deselected; 1 warning; 6.23 s	26 October 2026 terminal screenshots; earlier source/build.
Historical integration-marker run	194 passed; 521 deselected; 1 warning; 30.52 s	26 October 2026 terminal screenshots; earlier source/build.
Historical Postman Runner	90 requests; 470 assertions passed; 0 assertion failures; 0 execution errors	27 October 2026; earlier exported collection revision; one iteration; 10 s 697 ms.
Latest full pytest regression	751 passed; 1 warning; 0 failures/errors/skips; 122.67 s; exit 0	28 October 2026; retained text/XML and 74 source hashes.
Manual UAT	8 Pass; 12 Executed — Partially Verified; 0 Fail; 0 Not Executed	4–5 November 2026; 55 case-level browser captures.

Counts are not additive across runs. The historical 715-item full suite contains 113 unit + 194 integration + 408 other nodes; the 751-item run contains 116 unit + 227 integration + 408 other nodes. These are membership counts in complete runs, not separately executed marked selections.

8.1 Unit Test Report

Objective. Demonstrate isolated service and authentication rules and repository delegation.
Environment: Windows/win32; repository .venv; Python 3.14.0; pytest 9.1.1; pluggy 1.6.0; anyio 4.15.1; pytest-asyncio 1.4.0; pyproject.toml.

Actual command (from the retained start capture, run from the repository root):

```
$base = Join-Path $env:LOCALAPPDATA "MyAdvisor-A2-Pytest"
.\.venv\Scripts\python.exe -m pytest tests -m unit -v --basetemp="$base\unit"
```

Observed summary: 715 collected, 113 selected, 602 deselected; **113 passed, 1 warning in 6.23 s**. Figure 8.3 shows the genuine completed result. The warning is a StarletteDeprecationWarning about httpx use with starlette.testclient; it is a dependency-maintenance warning, not a failed assertion.

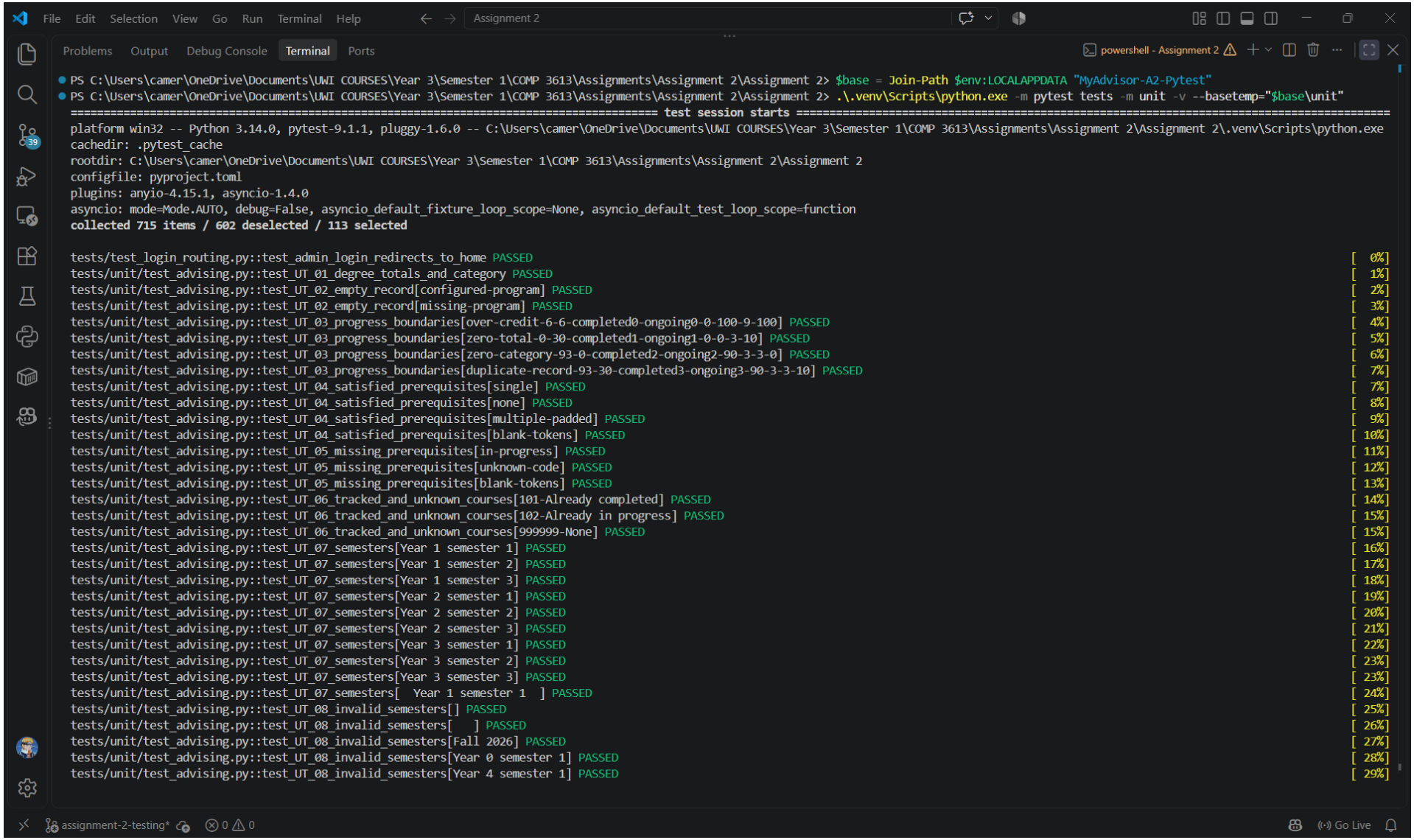


Figure 8.1. Retained start capture: command, environment, collection totals and early passing tests. 26 October 2026. Original terminal capture published as Screenshot 2026-10-26 231703.png.

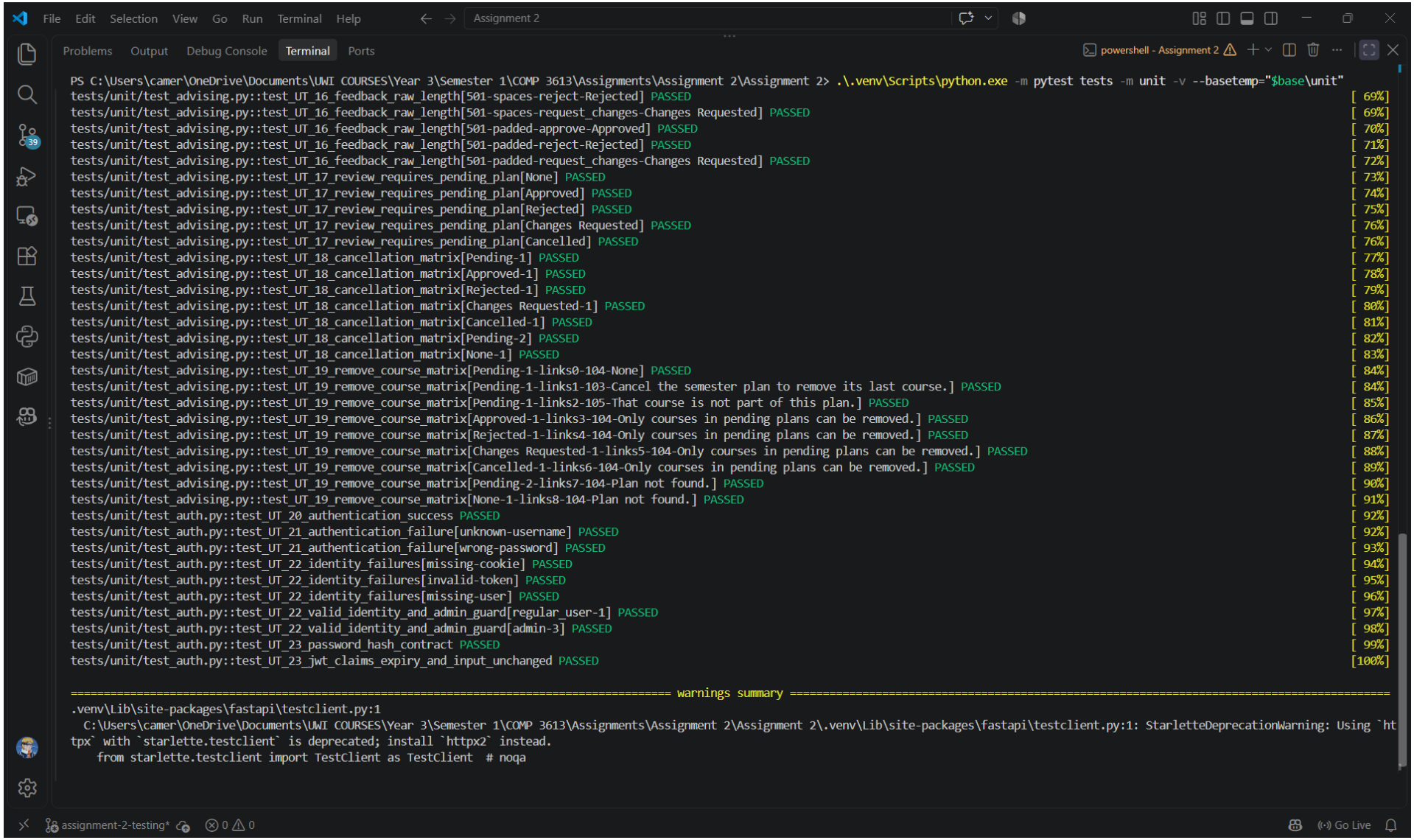


Figure 8.2. Retained later capture: later UT variants and the warning text. 26 October 2026. Original terminal capture published as Screenshot 2026-10-26 231742.png.

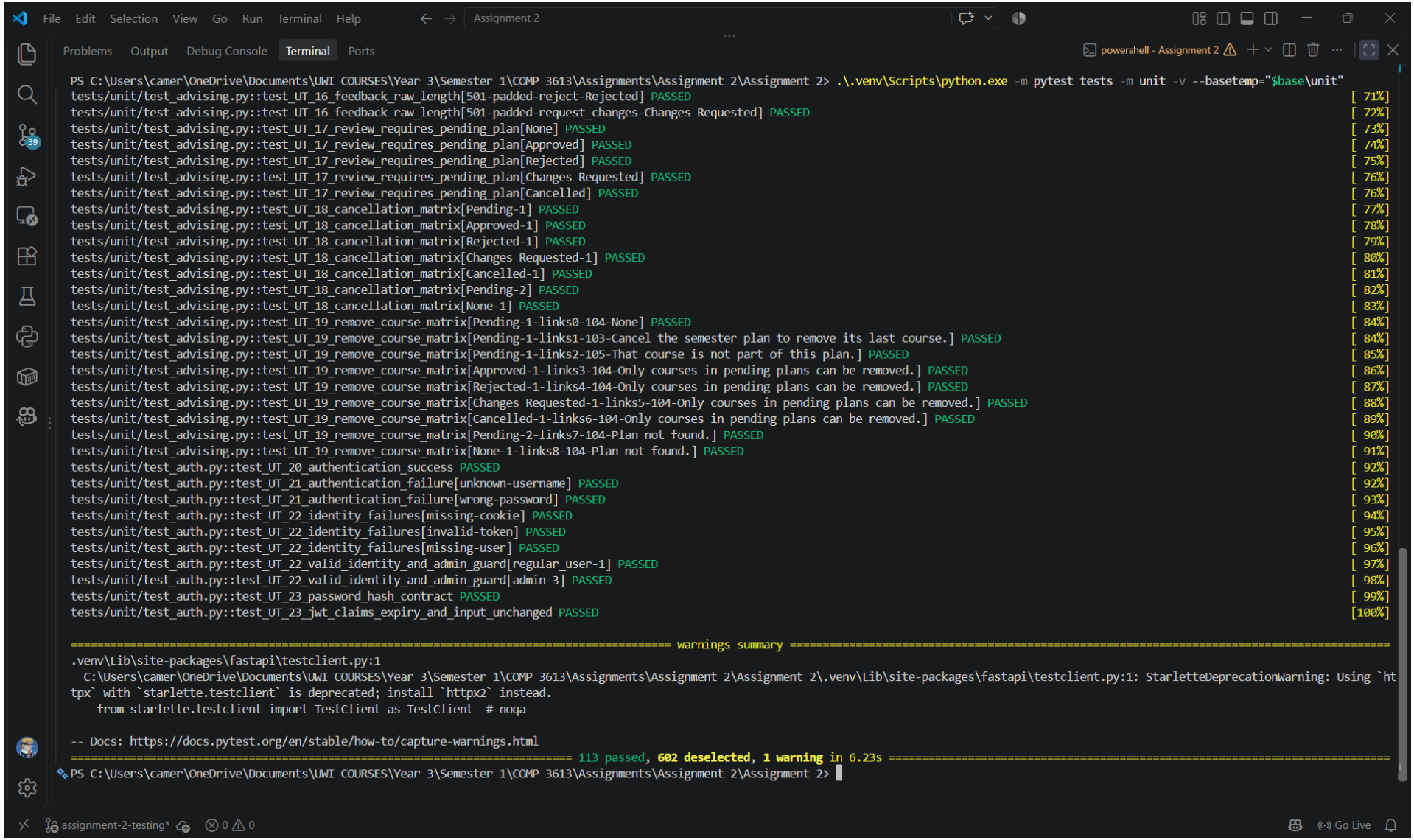


Figure 8.3. Completed unit result: **113 passed, 602 deselected, 1 warning in 6.23 s**. 26 October 2026. Original terminal capture published as Screenshot 2026-10-26 231747.png.

8.2 Integration Test Report

Objective. Verify that real authentication, role/ownership dependencies, HTTP parsing, Student/Advisor workflows and database persistence work together. Components include TestClient, FastAPI, services and repositories, and per-case SQLModel/SQLite with independent request sessions; database-only cases reach real constraints and rollback. **Environment:** Windows/win32; Python 3.14.0; pytest 9.1.1; pluggy 1.6.0; anyio 4.15.1; pytest-asyncio 1.4.0; repository .venv.

Actual command (same writable temporary base defined in the unit start capture):

```
.\.venv\Scripts\python.exe -m pytest tests -m integration -v --basetemp="$base\integration"
```

Observed summary: 715 collected, 194 selected, 521 deselected; **194 passed, 1 warning in 30.52 s**. Figure 8.6 shows the genuine completed result. The warning is the same disclosed TestClient/httpx deprecation. The selected cases cover queue receipt, approve/reject/request-changes decisions, feedback delivery, same-ID resubmission, removal, allowed and forbidden cancellation, replacement history, cross-user privacy, missing/invalid authentication, validation and fresh-session state assertions.

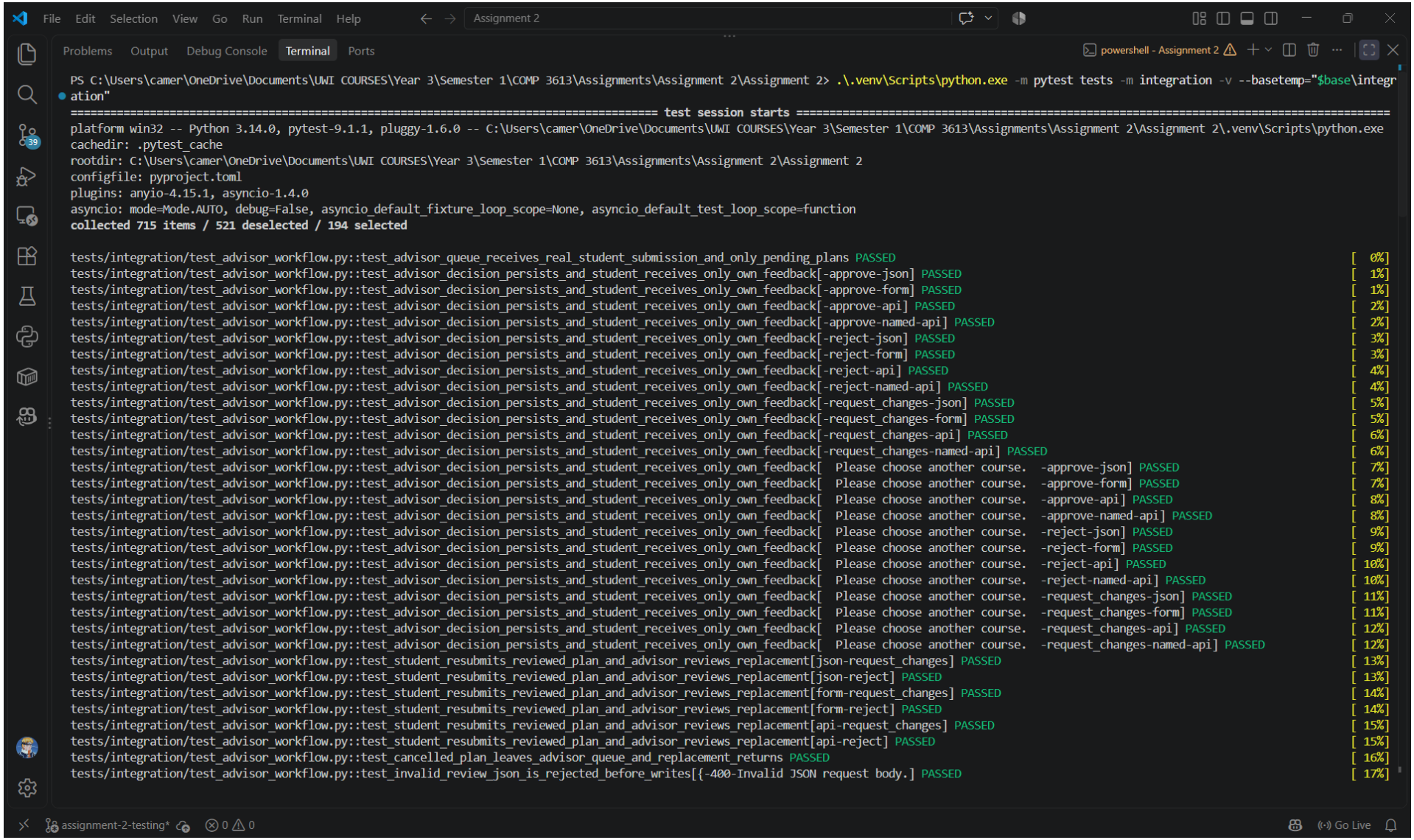


Figure 8.4. Retained start capture: command, environment, queue receipt, Advisor lifecycle and Student variants. 26 October 2026. Original terminal capture published as Screenshot 2026-10-26 231956.png.

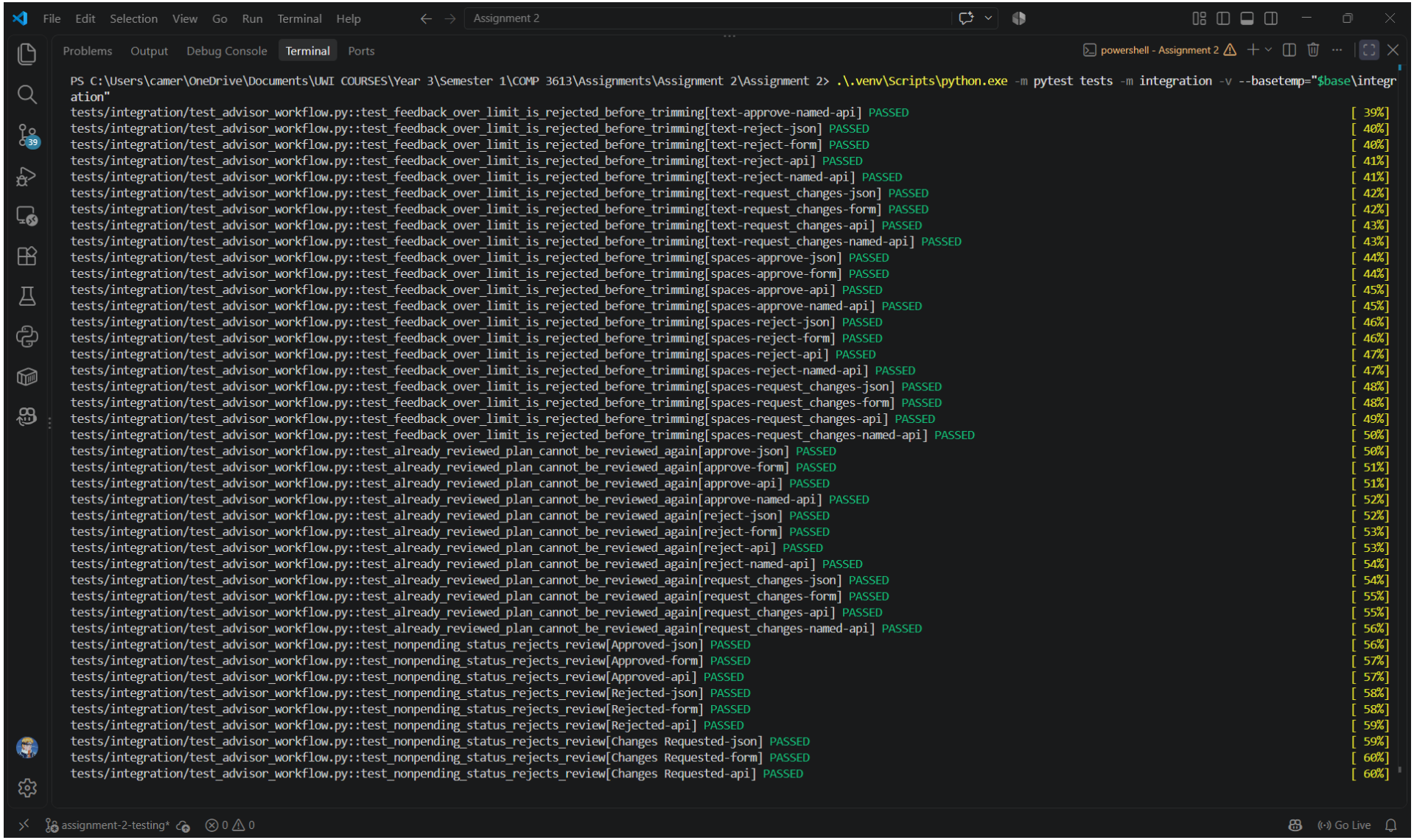


Figure 8.5. Retained later capture: Advisor validation, feedback boundaries and repeated-review variants. 26 October 2026. Original terminal capture published as Screenshot 2026-10-26 232025.png.

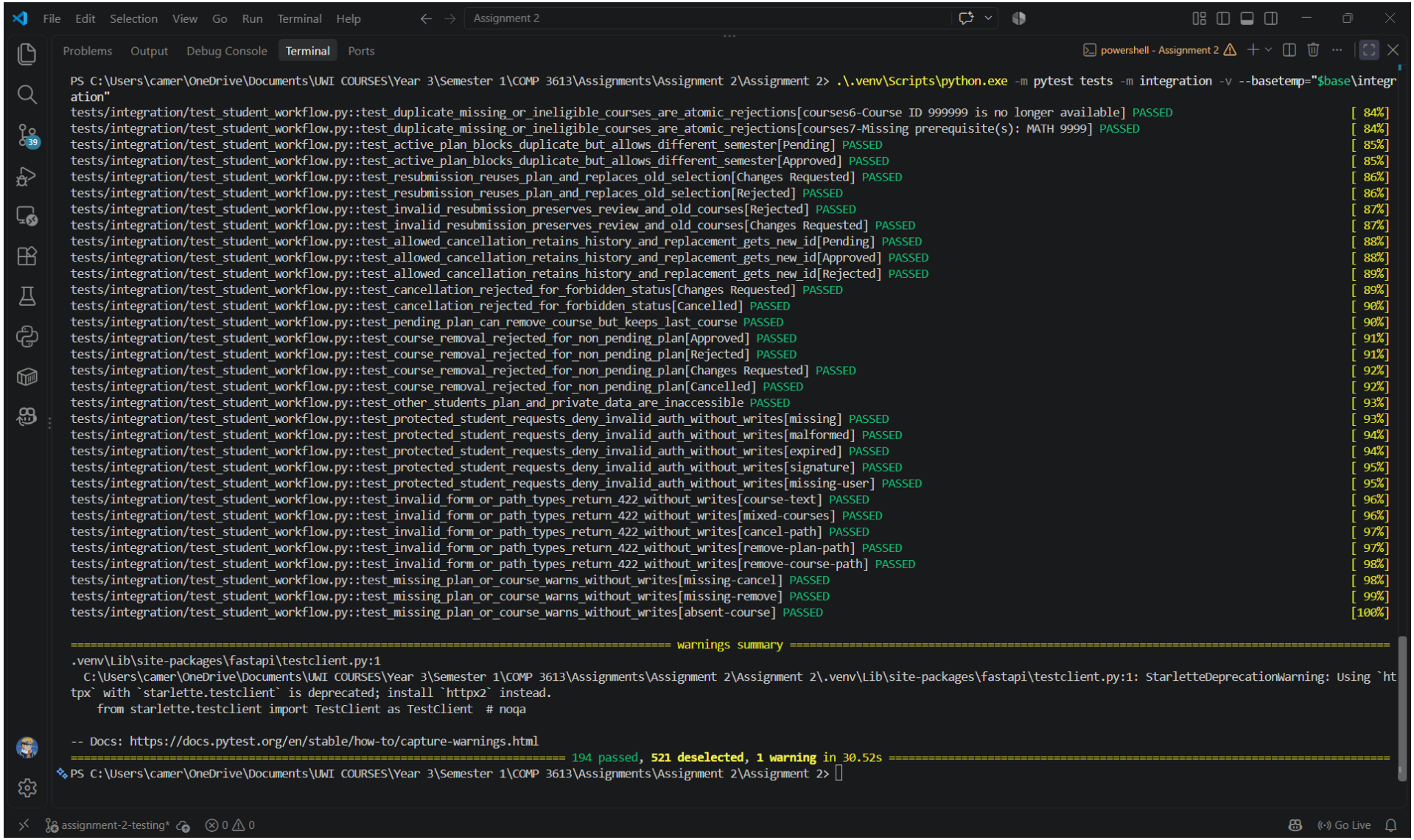


Figure 8.6. Completed integration result: **194 passed, 521 deselected, 1 warning in 30.52 s**. 26 October 2026. Original terminal capture published as Screenshot 2026-10-26 232058.png.

8.2.1 Latest verified full-suite regression

The latest verified complete pytest regression is separate from the historical marked runs. It contains **751 passed, 1 warning, and zero failures, errors or skips in 122.67 s**, exit code 0. The retained run metadata records 116 unit / 227 integration / 408 other nodes as membership in one complete run, not as new separate marked executions.

Measure	Verified value
Passed	751
Failed / errors / skipped	0 / 0 / 0
Warnings	1 (Starlette/httpx deprecation)
Duration	122.67 s
Exit code	0
Retained evidence	docs/assignment2/evidence/degree-requirements-2026-10-27/full-20261028T015858Z-e5cbdcfb.*
Source hashes	74 application/test SHA-256 entries, checked against the current files

Actual full-run command from the retained metadata (absolute paths abbreviated):

```
python -m pytest tests -q -p no:cacheprovider
  --basetemp=[unique disposable path] --junitxml=[retained full-run XML]
```

Verbatim retained final text line, not a screenshot:

```
751 passed, 1 warning in 122.67s
```

The run used a fresh writable temporary directory and did not touch a normal or private database. Corrected-build environment metadata records FastAPI 0.141.1, SQLModel 0.0.42, SQLAlchemy 2.0.54, PyJWT 2.14.0, pwdlib 0.3.1 and SQLite 3.50.4.

8.3 Postman Collection Run Report

Collection: MyAdvisor - COMP 3613 Assignment 2, v2.1 export. **Objective:** exercise all 15 JSON operations and supporting real login, registration and logout through the dependent Student/Advisor plan lifecycle, including negative status, role and ownership requests.

Execution detail	Observed / qualified value
Evidence date	27 October 2026 (user-confirmed; the Runner clock shows 01:25:19 AM without an independent time-zone record).
Environment	Postman Collection Runner; environment none; 1 iteration; duration 10 s 697 ms; average response time 29 ms.
Requests / assertions	**90 requests executed**; **470 assertions passed**; **zero observed assertion failures**; **zero execution errors**.
Authentication / authorization	Separate Student, Advisor and independent Student logins; expected anonymous and wrong-role denials; foreign ownership concealment and no-write read-back; response privacy.
Workflow scenarios	Progress/catalogue; invalid and valid submission; Pending queue; feedback limits; request changes and resubmit; reject and approve; immutable states; removal, cancellation, history, replacement and logout.
Conclusion	The recorded historical iteration passed its assertions. The published degree93 revision passed static validation but has not been rerun in Postman.

The captures below are genuine historical Runner views of the earlier exported collection revision. They show the same completed run across overlapping scroll positions, not six separate runs. They are not presented as execution of the current degree93 JSON.

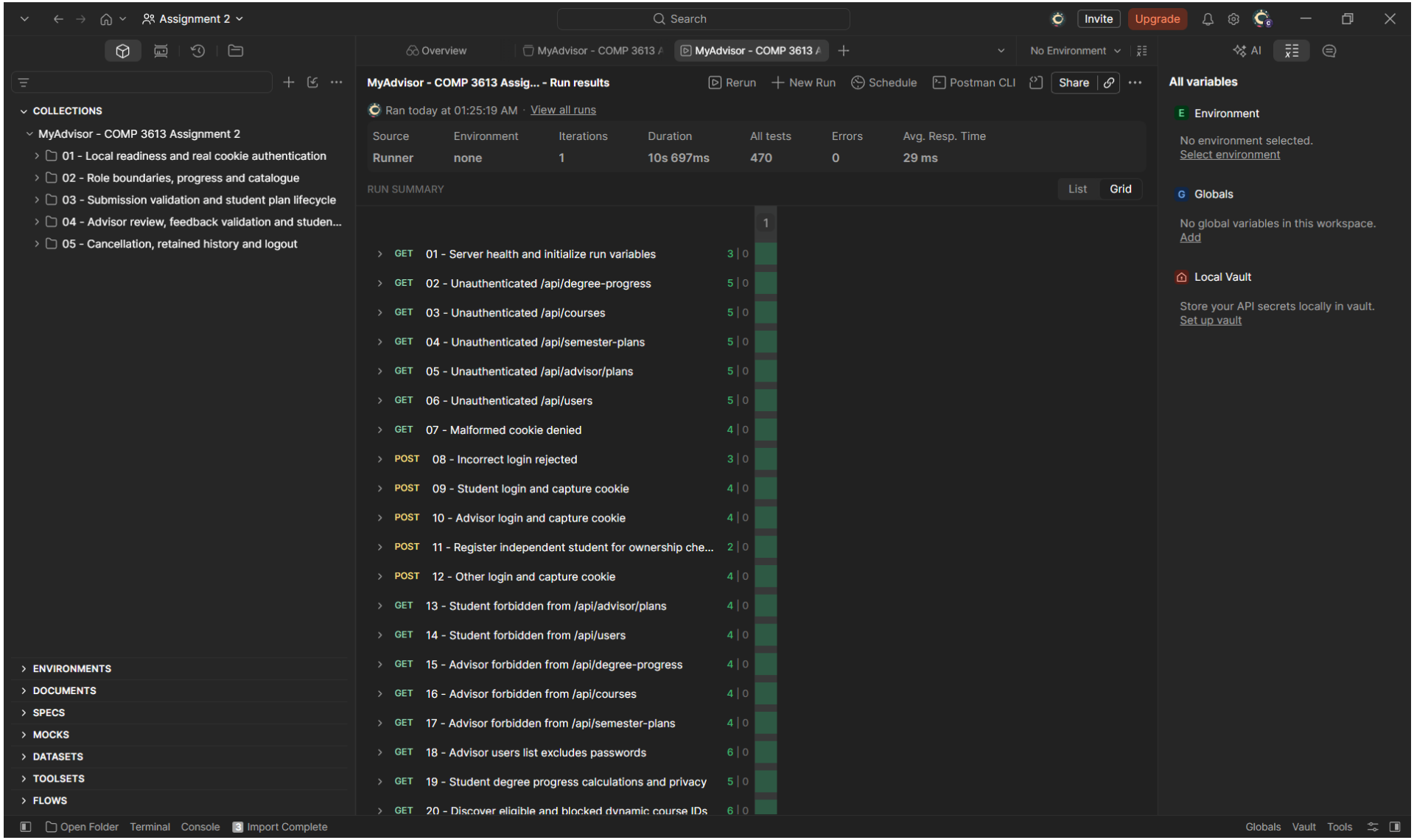


Figure 8.7. Runner capture: requests 01–19 visible; authentication, role boundaries and progress. Completed summary shows 470 assertions and zero errors. Historical collection revision; 27 October 2026.

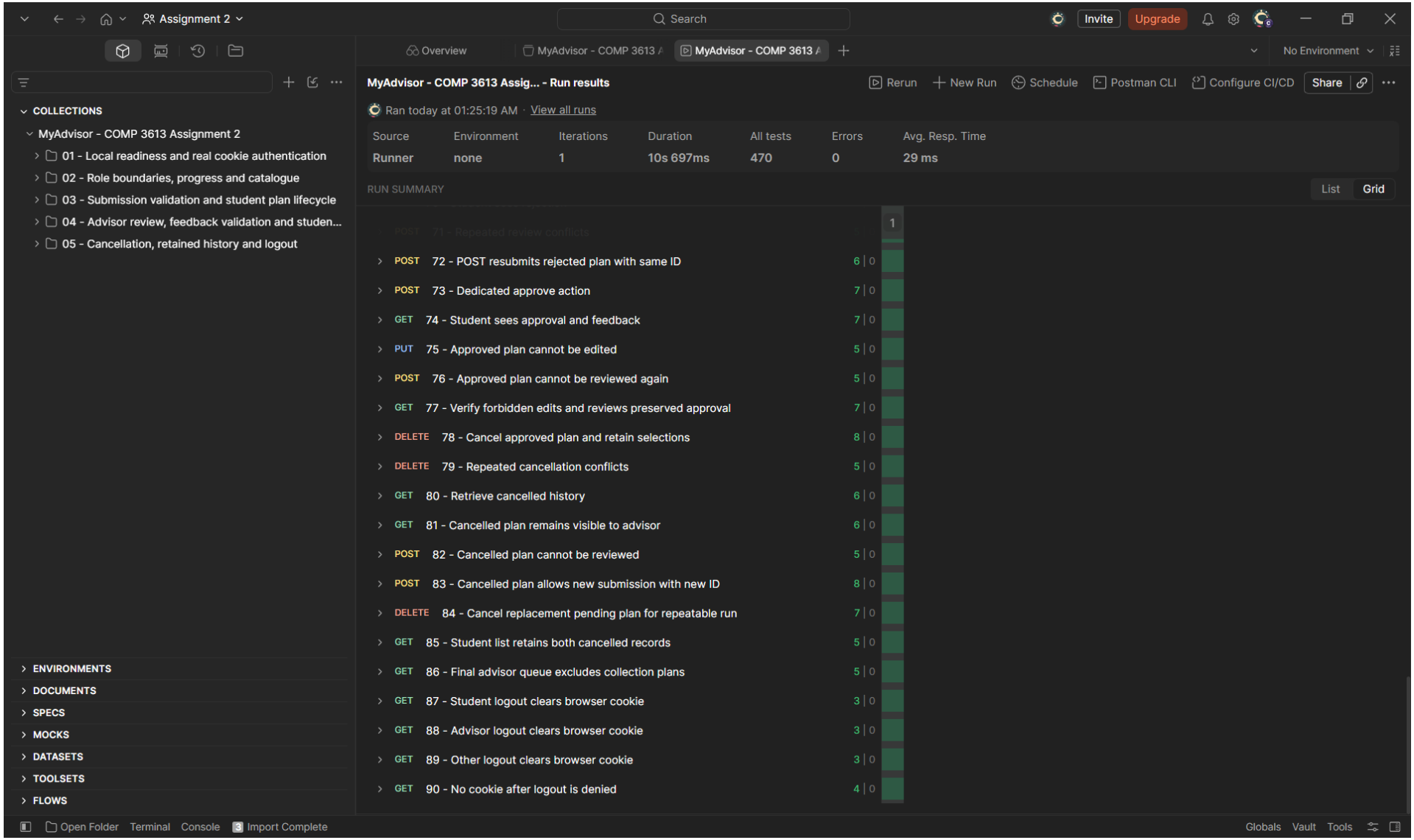


Figure 8.8. Runner capture: requests 72–90 visible; approved-state guards, cancellation/history, replacement and logout. The same completed summary is visible. Historical collection revision; 27 October 2026.

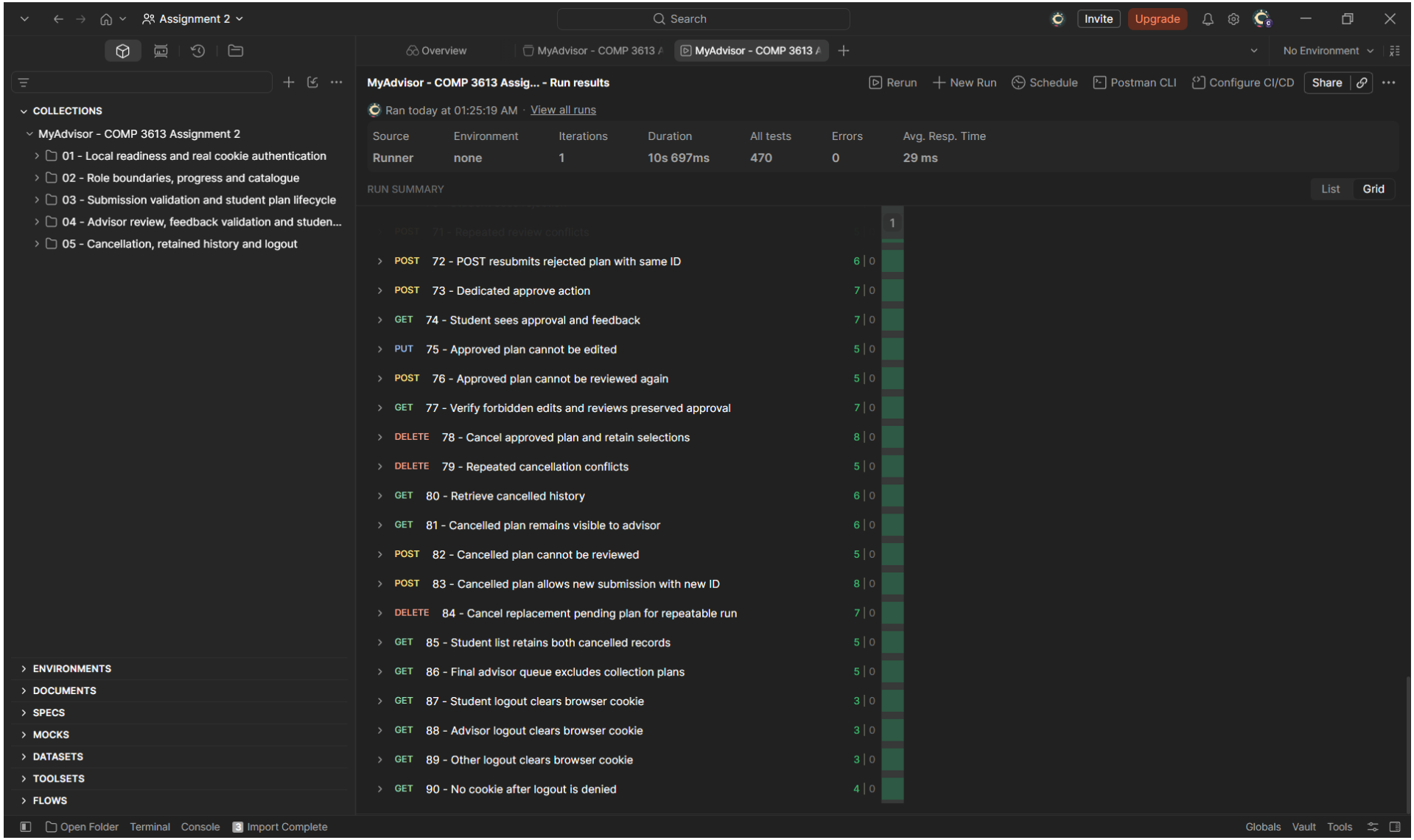


Figure 8.9. Additional summary view of requests 72–90 and the completed run: 90 requests, 470 assertions, zero assertion failures and zero execution errors. Historical collection revision; 27 October 2026.

9. UAT Execution Summary

The latest manual UAT execution used **55 case-level browser screenshots covering all 20 cases**. The tester was Cameron Ramesar; the application date shown in the captures is 5 November 2026, and the local files were saved on 4 November 2026 (UTC−04:00). No browser workflow, pytest run or Postman run was rerun to prepare this summary.

Result	Count	Cases
PASS	**8**	UAT-04, UAT-05, UAT-06, UAT-10, UAT-14, UAT-15, UAT-16, UAT-17
EXECUTED — PARTIALLY VERIFIED	**12**	UAT-01, UAT-02, UAT-03, UAT-07, UAT-08, UAT-09, UAT-11, UAT-12, UAT-13, UAT-18, UAT-19, UAT-20
FAIL	**0**	—
NOT EXECUTED	**0**	—

The 12 partially verified cases are **not failures**. Each has genuine execution evidence demonstrating its principal workflow; the remaining supplementary checks listed below were not captured or are not independently proven. No case is wholly unexecuted.

Observed execution data. Plan A used COMP 1201 + MATH 2250 (2 courses/6 credits); revised A used COMP 1201 + ENGL 1000 (2/6); B used MATH 2250 (1/3); C used COMP 1201 + MATH 2250 and then COMP 1201 only; replacement C used ENGL 1000 only (1/3). Degree Progress categories total 93 (3/36, 3/18, 0/12, 0/18, 0/9), with 6 completed, 3 in progress, 84 remaining, 93 required and 6%.

9.1 Results matrix

PASS — complete required outcome observed.

Case	Workflow	Scenario	Shots
UAT-04	WF2	Submit plan A	4
UAT-05	WF2	Submission status after navigation and relogin	6
UAT-06	WF3	Advisor pending review	2
UAT-10	WF2	Separate semester plan	4
UAT-14	WF2	Cancel Approved plan	6
UAT-15	WF2	Cancel Rejected plan	4
UAT-16	WF2 / WF3	Cancel Pending plan and clear the queue	6
UAT-17	WF2 / WF3	Start a new plan after cancellation	6

EXECUTED — PARTIALLY VERIFIED — principal workflow demonstrated; supplementary checks unconfirmed.

Case	Workflow	Scenario	Shots	Remaining supplementary checks
UAT-01	WF1	Degree progress totals and category breakdown	4	Refresh persistence is not independently captured (byte-identical pair).
UAT-02	WF2	Catalogue search and filters	6	Title search, semester switching, combined filters and the clear-filter state are not captured.
UAT-03	WF2	Eligibility and selection review	8	Zero-selection prevention and the full count sequence are not captured; ineligible examples differ from the plan.

Case	Workflow	Scenario	Shots	Remaining supplementary checks
UAT-07	WF3 / WF2	Request changes and feedback	6	Advisor action confirmation and removal from the Pending queue are not captured.
UAT-08	WF2 / WF3	Revise and resubmit	6	Current Pending Messages with cleared prior feedback is not captured.
UAT-09	WF3 / WF2	Approve revised plan	6	Advisor action confirmation and queue removal are not captured.
UAT-11	WF3 / WF2	Reject separate plan	6	Advisor action confirmation and queue removal are not captured.
UAT-12	WF2 / WF3	Pending plan for cancellation	4	A Approved and B Rejected are not co-visible with the C state.
UAT-13	WF2 / WF3	Remove course and protect the last course	4	The remove-confirmation dismissal/no-change state is not captured.
UAT-18	Login	Student login and clean-run baseline	6	Empty A/B/C Plan Courses and Messages baselines are not captured; the Home refresh pair is byte-identical.
UAT-19	Login / WF3	Advisor login	6	The refresh pair is byte-identical; a simultaneous separate Student session is not captured.
UAT-20	Login / security	Role and signed-out restrictions	10	The Student and Advisor signed-out checks share one byte-identical 401 image.

Four before/after pairs are byte-identical and therefore do not independently prove persistence or separate-role checks: UAT-01 degree progress/refresh, UAT-18 baseline/home, UAT-19 Advisor home/refresh, and UAT-20 Student/Advisor signed-out 401. The original files are preserved and are not presented as independent captures.

9.2 Evidence index

The complete 55-screenshot set is published in the repository at docs/assignment2/screenshots/UAT/. The table lists the case-level files referenced by the execution record; the folder also retains byte-identical renamed copies used for presentation, and no original file was deleted or modified.

Case	Published case-level evidence files
UAT-01	UAT-01-degree-progress.png UAT-01-after-refresh.png
UAT-02	UAT-02-search.png UAT-02-department.png UAT-02-no-results.png
UAT-03	UAT-03-empty-selection.png UAT-03-ineligible-courses.png UAT-03-review-two-courses.png UAT-03-selection-preserved.png
UAT-04	UAT-04-submission-success.png UAT-04-pending-after-refresh.png
UAT-05	UAT-05-home-pending.png UAT-05-messages-pending.png UAT-05-after-relogin.png
UAT-06	UAT-06-advisor-pending-review.png
UAT-07	UAT-07-advisor-feedback.png UAT-07-student-message.png UAT-07-editable-plan.png
UAT-08	UAT-08-resubmitted-pending.png UAT-08-revised-review.png UAT-08-advisor-revised-queue.png

Case	Published case-level evidence files
UAT-09	UAT-09-advisor-approved.png UAT-09-approved-after-refresh.png UAT-09-student-approved-message.png
UAT-10	UAT-10-semester1-still-approved.png UAT-10-semester2-pending.png
UAT-11	UAT-11-advisor-rejected.png UAT-11-student-rejected.png UAT-11-rejected-editable.png
UAT-12	UAT-12-advisor-pending.png UAT-12-student-pending.png
UAT-13	UAT-13-one-course-remaining.png UAT-13-advisor-updated.png
UAT-14	UAT-14-approved-cancelled.png UAT-14-cancellation-dismissed.png UAT-14-messages-cancelled.png
UAT-15	UAT-15-rejected-cancelled.png UAT-15-messages-cancelled.png
UAT-16	UAT-16-pending-cancelled.png UAT-16-messages-cancelled.png UAT-16-advisor-queue-cleared.png
UAT-17	UAT-17-new-selection-review.png UAT-17-new-pending-plan.png UAT-17-advisor-new-plan.png
UAT-18	UAT-18-invalid-login.png UAT-18-baseline.png UAT-18-home.png
UAT-19	UAT-19-invalid-login.png UAT-19-advisor-home.png UAT-19-after-refresh.png
UAT-20	UAT-20-student-401.png UAT-20-advisor-401.png UAT-20-student-403.png UAT-20-advisor-plan-fallback.png UAT-20-advisor-progress-fallback.png

9.3 Representative screenshots

The following captures are representative examples of the 55-image evidence set. Each caption states the case status. The complete set remains published in the repository; the PDF does not duplicate all 55 images in order to keep the submission compact and readable.

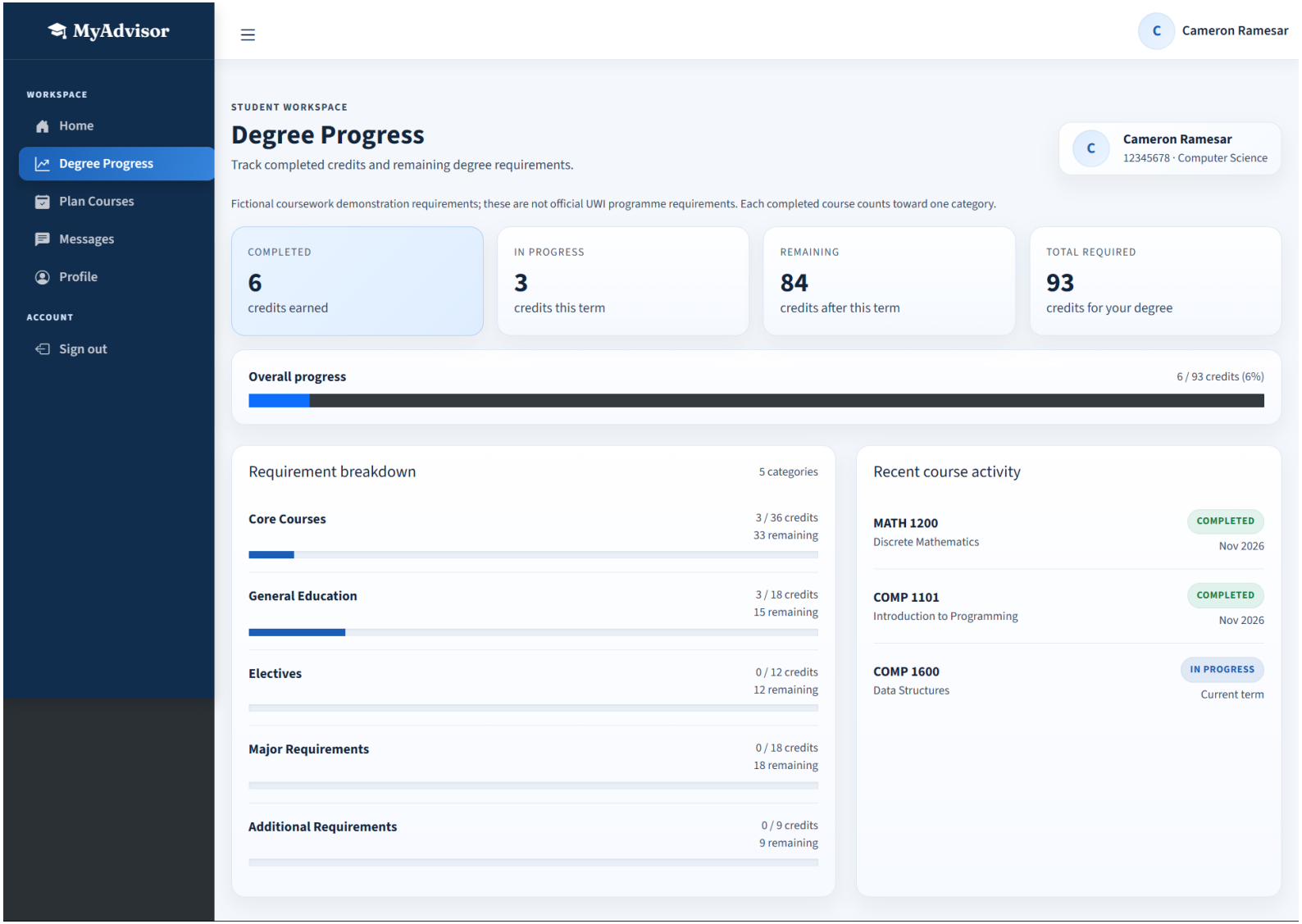


Figure 9.1. UAT-01 — View degree progress — status: **EXECUTED — PARTIALLY VERIFIED**. Degree Progress shows 6 completed, 3 in progress, 84 remaining and 93 required, with the corrected category breakdown 3/36, 3/18, 0/12, 0/18 and 0/9. Refresh persistence is not independently captured (duplicate pair). Original capture UAT-01-degree-progress.png; application date 5 November 2026 (local capture 4 November 2026).

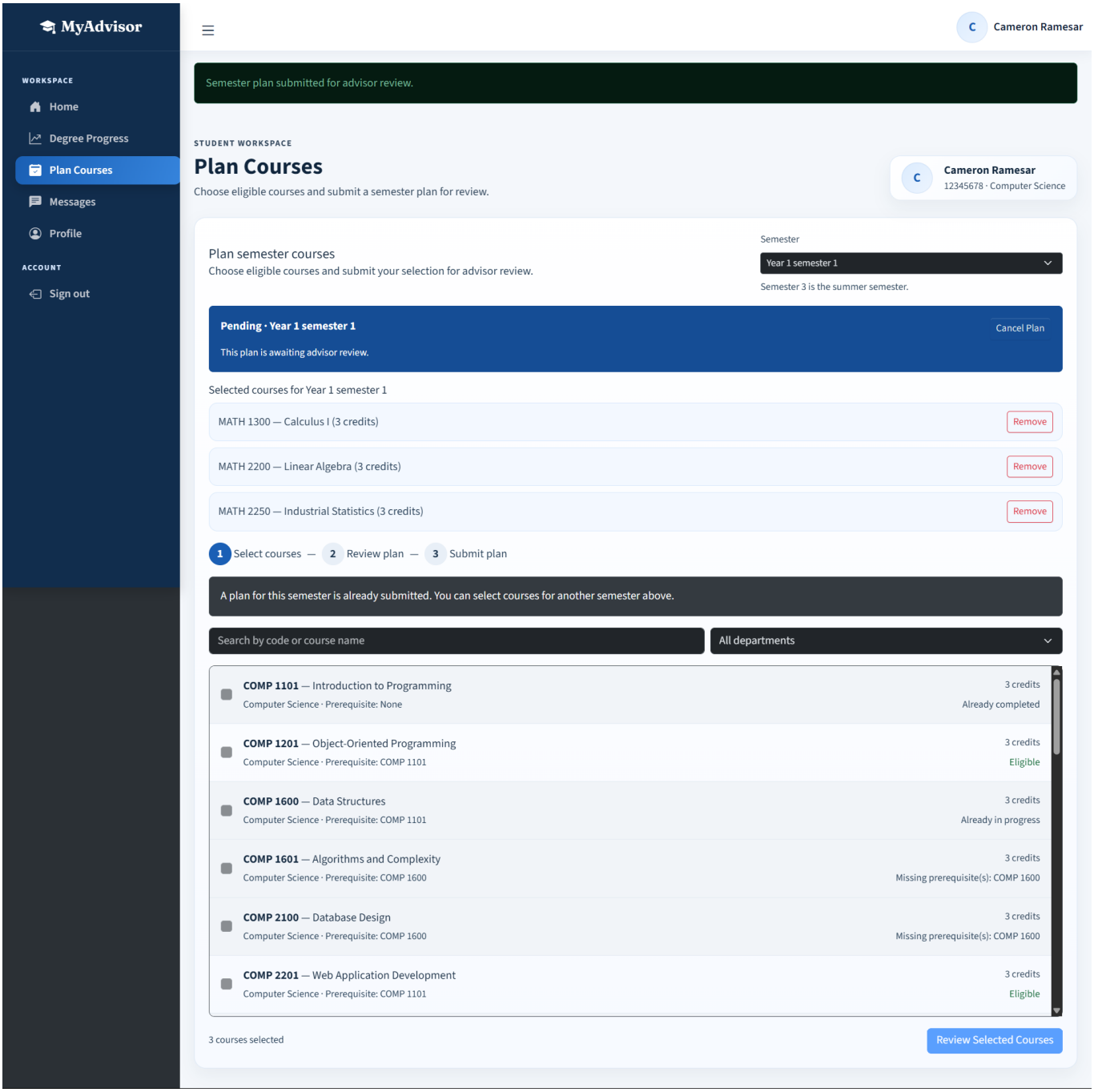


Figure 9.2. UAT-04 — Submit semester plan A — status: **PASS**. Plan A is Pending with COMP 1201 + MATH 2250 (2 courses/6 credits); the after-refresh capture shows the same Pending state and locked selection form. Original capture UAT-04.png; application date 5 November 2026 (local capture 4 November 2026).

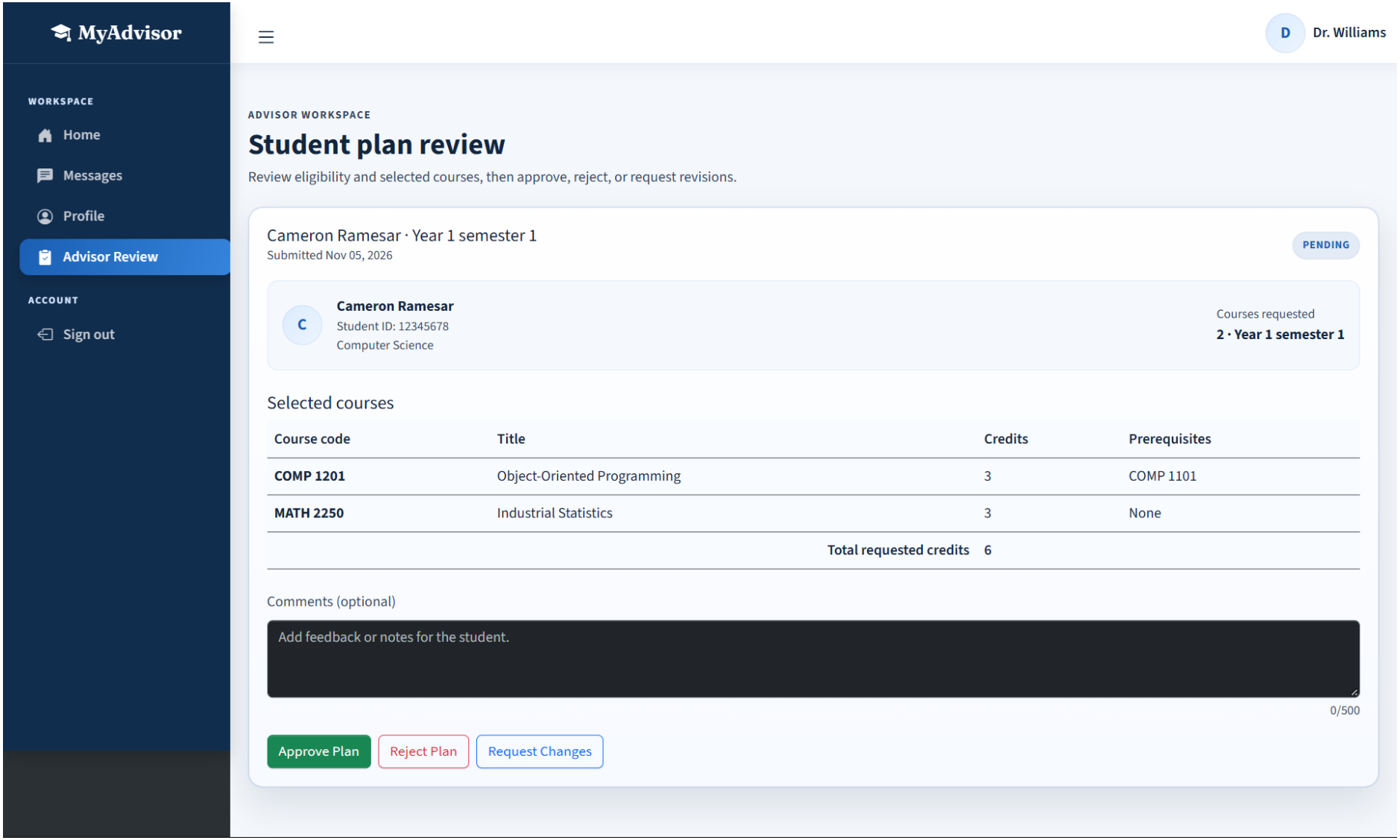


Figure 9.3. UAT-06 — Advisor reviews the submitted plan — status: **PASS**. Advisor Review shows the Student summary, Year 1 semester 1, PENDING, COMP 1201 + MATH 2250, 2 courses/6 credits and the Approve/Reject/Request Changes controls. Original capture UAT-06-advisor-pending-review.png; application date 5 November 2026 (local capture 4 November 2026).

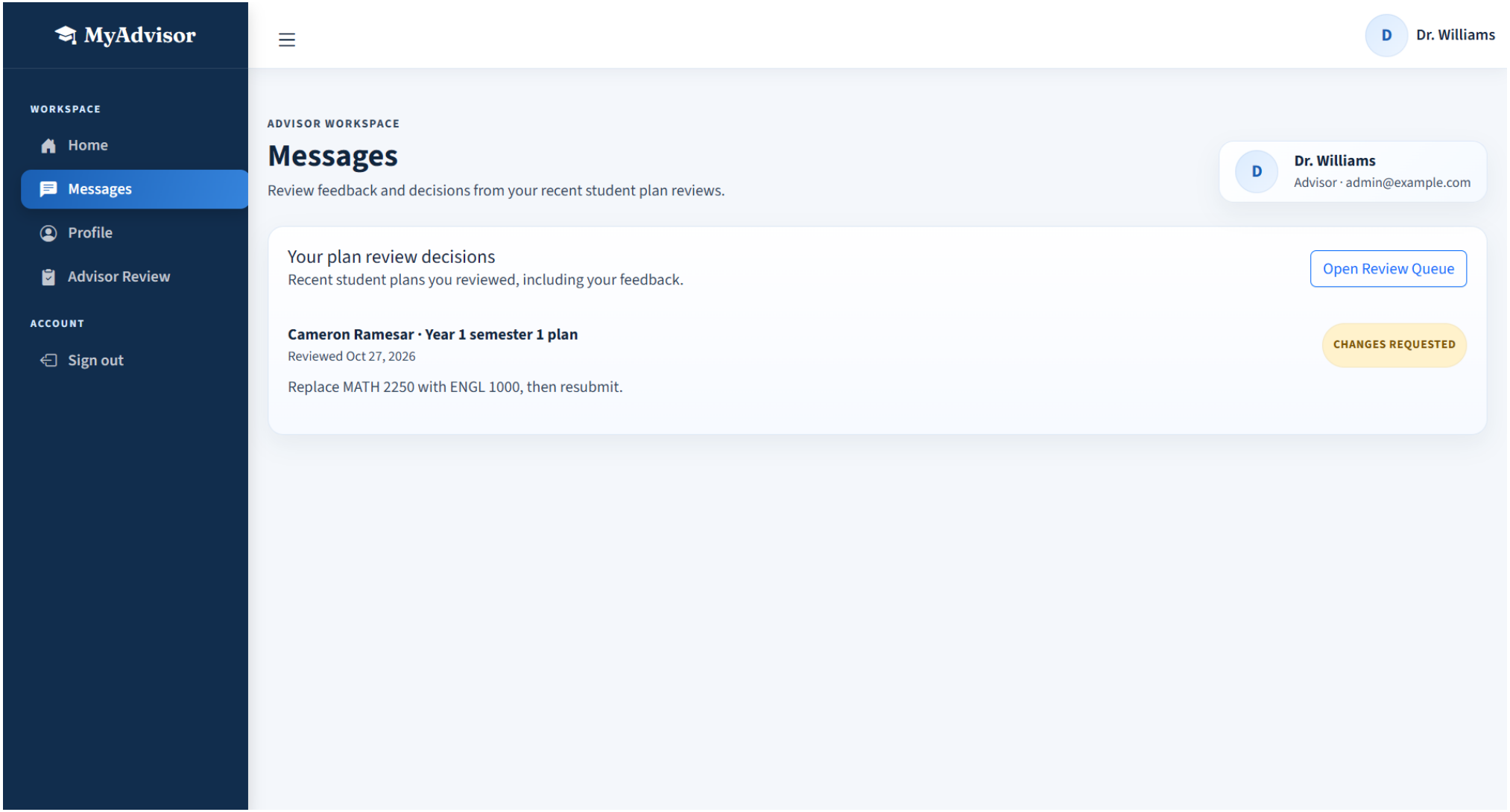


Figure 9.4. UAT-07 — Request changes and deliver feedback — status: **EXECUTED — PARTIALLY VERIFIED**. Advisor feedback is recorded for plan A; the Student sees the same feedback and the plan remains editable. Advisor action confirmation and queue removal are not captured. Original capture UAT-07-request-changes.png; application date 5 November 2026 (local capture 4 November 2026).

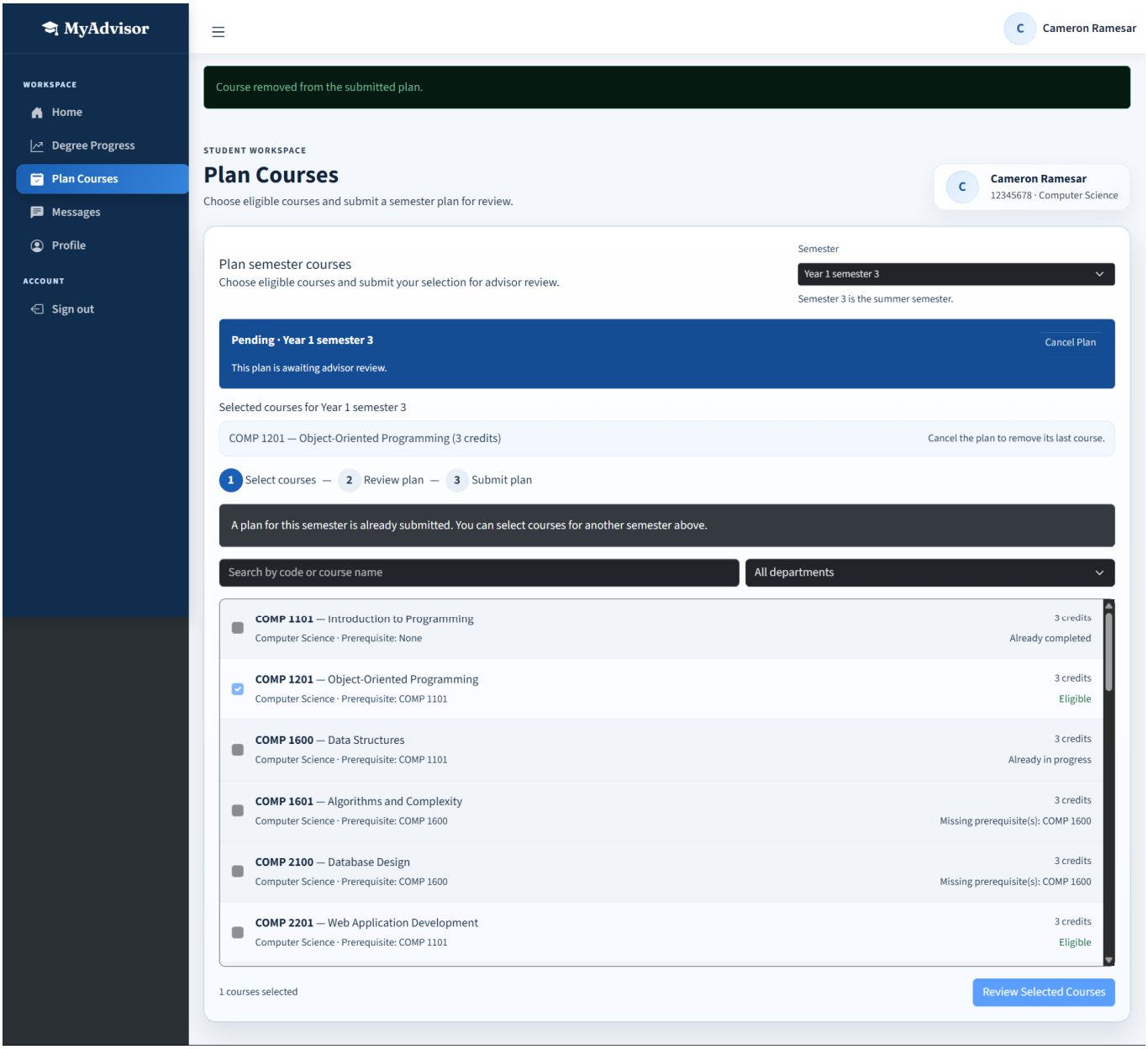


Figure 9.5. UAT-13 — Remove a course and protect the last course — status: **EXECUTED — PARTIALLY VERIFIED**. Plan C remains Pending with COMP 1201 only (1 course/3 credits), and the final-course guard is visible. The remove-confirmation dismissal state is not captured. Original capture UAT-13-course-removed.png; application date 5 November 2026 (local capture 4 November 2026).

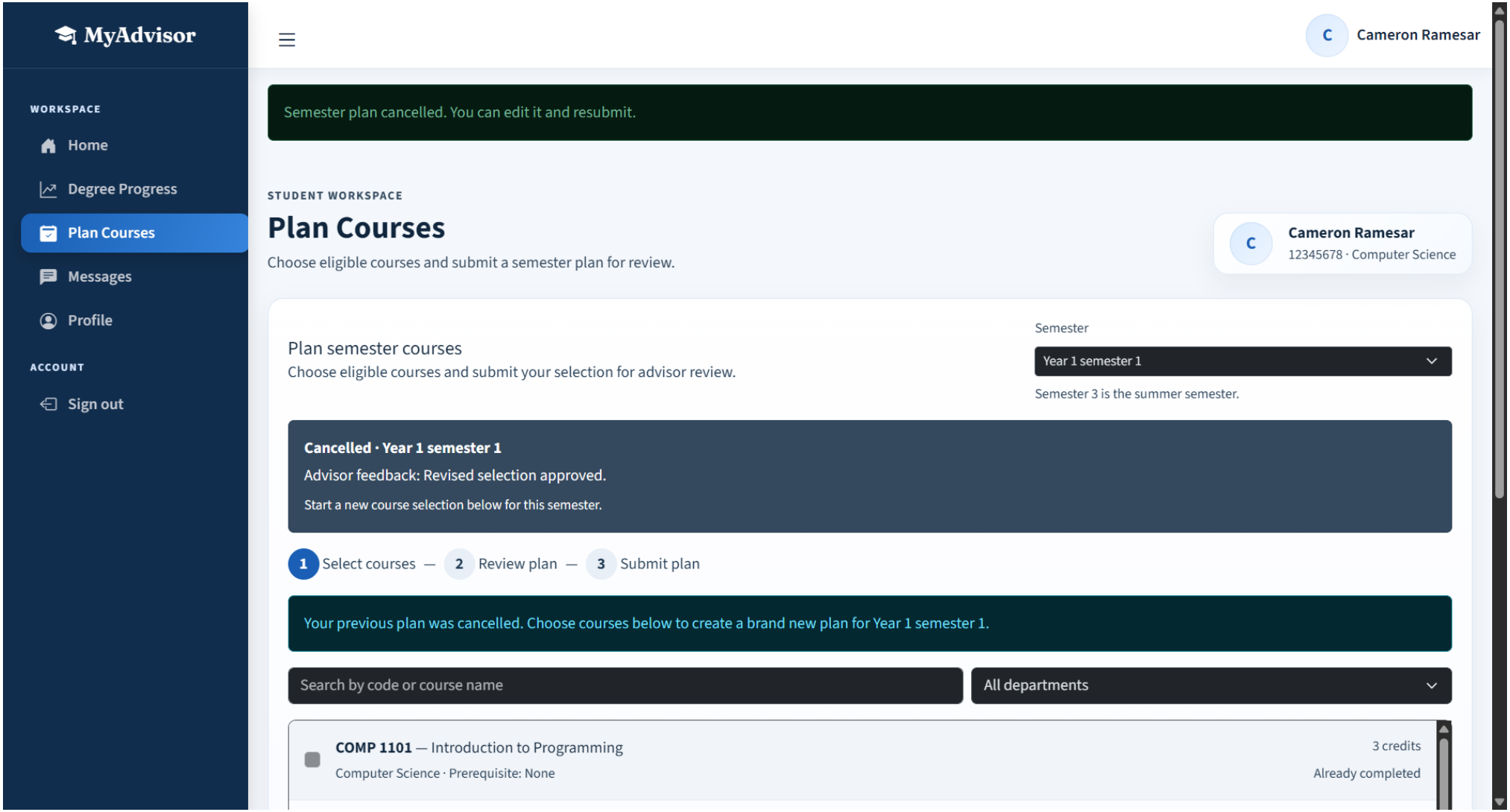


Figure 9.6. UAT-14 — Cancel an Approved plan — status: **PASS**. Plan A is Cancelled with the prior approval feedback retained and zero selected; the dismissed-confirmation capture still shows A Approved. Original capture UAT-14-approved-cancelled.png; application date 5 November 2026 (local capture 4 November 2026).

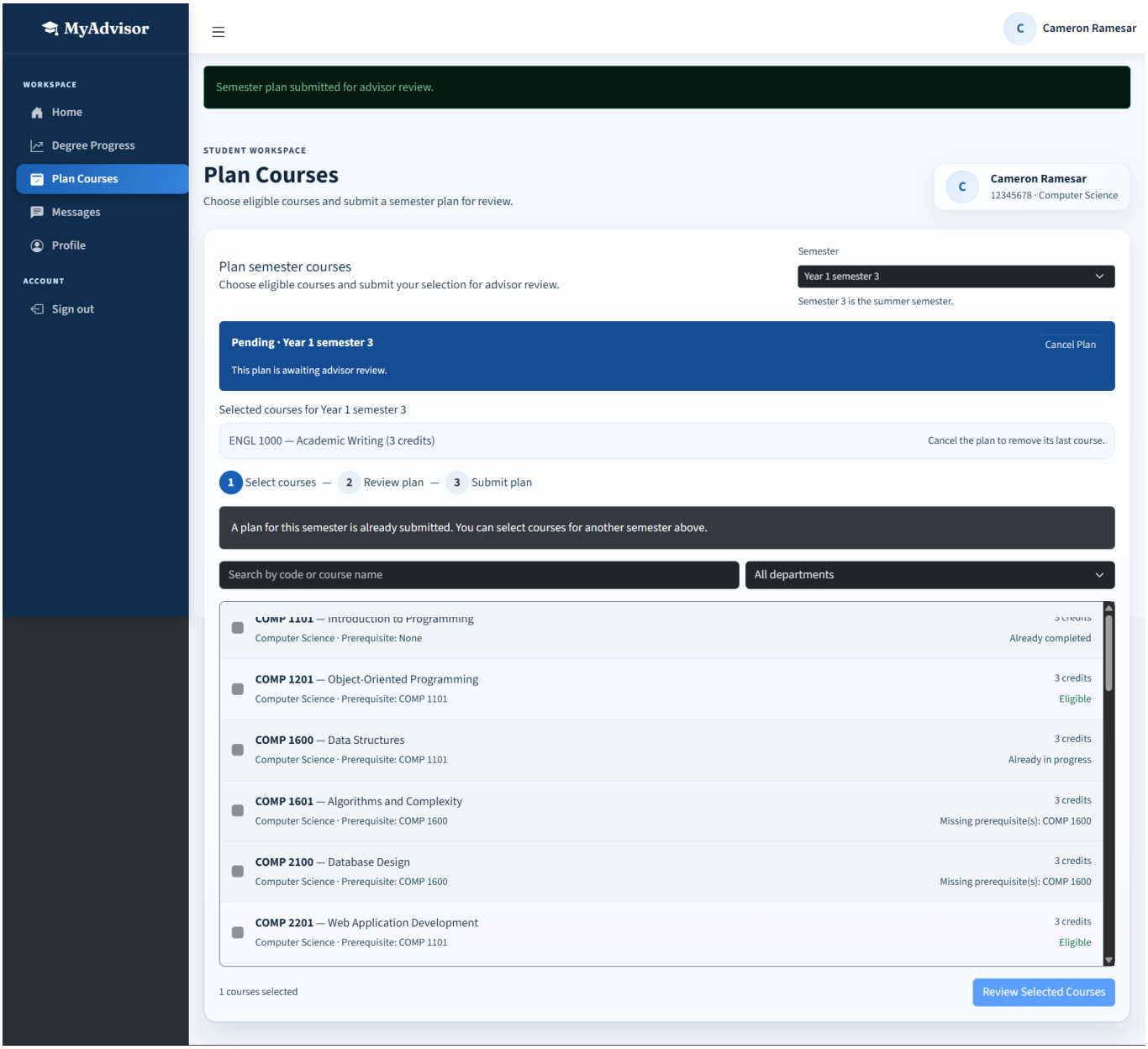


Figure 9.7. UAT-17 — Start a new plan after cancellation — status: **PASS**. After cancellation, plan C is replaced with a new Pending plan containing ENGL 1000 only (1 course/3 credits); the Advisor queue shows the replacement. Original capture UAT-17-new-plan.png; application date 5 November 2026 (local capture 4 November 2026).

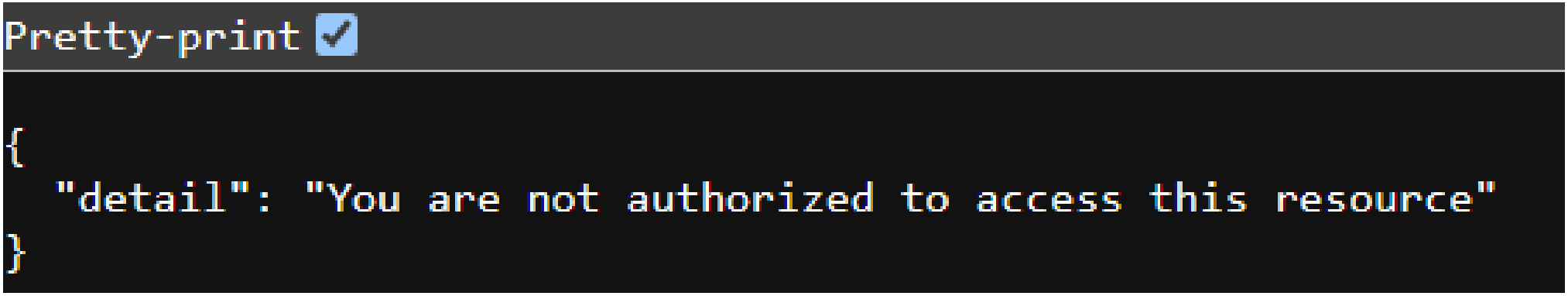


Figure 9.8. UAT-20 — Role and signed-out restrictions — status: **EXECUTED — PARTIALLY VERIFIED**. A logged-in Student receives the 403 JSON authorization response for an Advisor/admin resource. The separate Student and Advisor signed-out checks share one byte-identical 401 image. Original capture UAT-20-student-403.png; application date 5 November 2026 (local capture 4 November 2026). Shown as a cropped detail: the unused dark background is omitted for legibility and the original capture is unchanged.

Appendix A — Testing Limitations and Future Improvements

This appendix records the material limitations that qualify the evidence. It is not a second audit and does not change the academic deliverables. The project is complete as an academic submission; production deployment readiness is a separate question and is not claimed.

Area	Verified limitation	Effect on this submission
UAT supplementary checks	12 cases have genuine principal-workflow evidence but unverified supplementary checks: refresh persistence (duplicate pairs), empty A/B/C baselines, Advisor confirmation/queue removal, remove-confirmation dismissal, combined search/filter states, and separate post-logout role checks.	12 cases are classified Executed — Partially Verified, not Pass and not Fail. The principal workflows are demonstrated.
Postman runtime evidence	The 90-request / 470-assertion Runner result belongs to the earlier exported collection revision. The published degree93 revision passed static validation but has not been rerun in Postman.	The historical Runner evidence is reported as historical. No new Runner pass is claimed for the revised collection.
CSRF protection	No explicit CSRF token or Origin check was added to authenticated form writes; the production cookie is configured with SameSite=None and logout remains a GET request. No live exploit was executed.	Production security hardening remains a review item. It does not prevent academic submission, and no production certification is claimed.
Production signing key	The deployment blueprint requests a generated production key, but the live runtime configuration was not independently verified; settings still permit fallback to the public example key.	A production deployment must verify a strong, private signing key before use. No production key is reproduced in this document.
Historical build provenance	The unit, integration and Postman screenshots are genuine but tied to earlier source/build executions. The UAT session retained dated captures but not complete browser/version, URL, database path or build metadata.	The evidence supports the reported results but is not a cryptographic certification of every tested revision.
Database and deployment scope	Automated tests use isolated SQLite. PostgreSQL behaviour, concurrency, load, production cookies and deployed CSRF behaviour were not independently exercised.	The results establish the coursework application and its local workflows, not production-scale behaviour.
Dependency warning	One Starlette/httpx deprecation warning appears in the unit and integration runs. It is a dependency-maintenance warning, not a failed assertion.	All selected assertions passed; a future dependency upgrade should be regression-tested.

Future improvements. Complete the remaining UAT supplementary checks and record the session build/URL/database metadata; rerun the published degree93 Postman collection and archive the new Runner result; add and test CSRF/Origin protection for authenticated form writes; verify the production signing-key configuration and remove the public-key fallback in production; exercise the application against PostgreSQL and concurrent requests; and complete a human readability review of the accessible contrast states.

Readiness statement. The document contains every lecturer-required section and the genuine evidence needed for academic assessment. The limitations above are disclosed so that the report does not overstate production security or complete verification. They are not submission blockers for the coursework deliverables.

Appendix B — Submission Checklist

This checklist distinguishes the required PDF content, repository publication and disclosed testing limitations. It describes the academic submission, not production deployment readiness.

Required PDF content	Status
1. Name & ID	Included — professional cover page, Cameron Ramesar, 816044668
2. Project Description	Included — purpose, roles, WF1–WF3, technologies, 93-credit configuration
3. Use Case Diagram	Included — original diagram, Figure 3.1
4. Entity Relationship Diagram	Included — original ERD and readable keyed rendering, Figures 4.1–4.2
5. Application Wireframe	Included — original three-panel wireframe and three enlarged panels, Figures 5.1–5.4
6. Test Plans	Included — 6.1 UAT, 6.2 Unit, 6.3 Integration
7. Implementation	Included — 7.1 repository, 7.2 unit, 7.3 integration, 7.4 Postman
8. Reports	Included — 8.1 Unit, 8.2 Integration, 8.2.1 full regression, 8.3 Postman
UAT execution summary	Included — Section 9, 8 Pass / 12 Partial / 0 Fail / 0 Not Executed
Testing limitations	Disclosed — Appendix A
Required PDF content	**Included — no missing lecturer-required section**
Repository publication	**Verified — commit 847e39a9cc6340e51bcc05c9864f75f8457021c1; required paths, 55 UAT captures and 90-request degree93 JSON present**
Additional testing limitations	**Disclosed — Appendix A**
Academic submission	**Ready**
Production deployment readiness	**Not claimed**

Link verification. Repository: https://github.com/Camcoon/MyAdvisorWithTests (HTTP 200). The published Postman collection JSON is linked from Section 7.4 and returned HTTP 200. Verified on 4 November 2026.

Evidence integrity. The original unit, integration, Postman and UAT images were reused without modification. No application code, test, UAT or Postman result was rerun or fabricated for this document. Password constants and authentication-token values are not reproduced.